
Software Requirements Specification

for

GoWhere

Prepared by

Keith Lim En Kai (U2220506C)

Lee Seungju (U2221620J)

Jesica Tjan (U222911J)

Jerrell Yeoh Shao-En (U2220449D)

Lim Shaojun (U2221218F)

CodeCrafters

10 April 2024

Table of Contents

Table of Contents	2
1. Introduction	5
1.1 Purpose	5
1.2 Document Conventions	5
1.3 Intended Audience and Reading Suggestions	5
1.4 Product Scope	6
1.5 References	6
2. Overall Description	7
2.1 Product Perspective	7
2.2 Product Functions	7
2.3 User Classes and Characteristics	7
2.4 Operating Environment	9
2.5 Design and Implementation Constraints	10
2.6 User Documentation	11
2.7 Assumptions and Dependence	11
3. External Interface Requirements	13
3.1 User Interfaces	13
3.2 Hardware Interfaces	22
3.3 Software Interfaces	23
3.4 Communications Interfaces	23
4. System Features	23
4.1 Create Account	23
4.2 Account Login	26
4.3 Reset Password	27
4.4 View Account	28
4.5 Change Password	29
4.6 Random Search	30
4.7 View Map and Location Details	32
4.8 View More Reviews	33
4.9 View Current Weather	34
4.10 Save Locations	35
4.11 View Saved Locations	36
5. Other Non-functional Requirements	37
5.1 Performance Requirements	37
5.2 Security Requirements	37
5.3 Availability	37
5.4 Localization	37
5.5 Usability	38
5.6 Reliability	38

5.7 Maintainability	38
5.8 Testability	38
5.9 Reusability	38
7. Use Case Model	39
8. System Architecture - Lab 3 Deliverables	54
9. Other Requirements	55
Appendix A: Glossary	55
9.1 Data Dictionary	55
Appendix B: Analysis Models	56
9.2 Class Diagram (Conceptual Model)	56
9.3 Sequence Diagram (Dynamic Model)	57
9.4 Dialog Map (Dynamic Model)	66
9.5 Test Cases	67

Revision History

Name	Date	Reason For Changes	Version
Final	10 April 2024	Final Revision for SRS Document	1.0

Lab 4 Deliverables: (For TA and Lab Tech)

1. Working Application Prototype (in Code section of Github)
2. Source Code (in Code section of Github)
3. Test Cases and Testing Results (Section 9.5 of this Document)
4. Demo Script (Section 9.6 of this Document)

1. Introduction

1.1 Purpose

This Software Requirement Specification (SRS) document is created for the “GoWhere” web application, version 1.0. The primary goal of this SRS document is to outline the requirement specifications for the GoWhere web application, streamlining the development and production processes for all stakeholders involved. Every aspect including: system features, limitations, non-functional requirements and interface specifications of the web application is documented within this SRS document.

1.2 Document Conventions

This section describes the conventional standards used throughout this document. All readers must pay attention to the standards listed in this section.

- Font: Arial
- Heading 1: Size 18
- Heading 2: Size 14
- Heading 3: Size 12
- Normal Text: Size 11
- Technical Standards: IEEE 830-1998

Refer Appendix A: Data Dictionary for the definitions of special terms used throughout this documentation.

1.3 Intended Audience and Reading Suggestions

This document is designed for various stakeholders, encompassing GoWhere web application users, the GoWhere development and testing teams, project managers, and the GoWhere marketing team. It commences by articulating the purpose of the GoWhere web application and outlining conventions applied throughout. Following this, it presents a high-level overview of the application's functionalities, along with design constraints and assumptions. Subsequently, the document delineates the interface requirements. Finally, it provides an in-depth exploration of the system features and non-functional requirements.

All stakeholders are advised to initiate their review with Section 1.1 (Purpose), 1.2 (Document Conventions), and Appendix A (Data Dictionary) to familiarize themselves with the web application's purpose, documentation standards, and technical term definitions.

The GoWhere development team is strongly urged to delve into Section 2 (Overall Description) for a comprehensive understanding of application functionalities, design, and constraints. Following this, Section 4 (System Features) provides developers with insights into each system feature to be incorporated. Lastly, developers should consult Section 3 (External Interfaces Requirements) and 5 (Other Nonfunctional Requirements) to grasp the specified requirements for the application's optimal functioning.

Conversely, users of the GoWhere web application, the GoWhere testing team, project managers, and the GoWhere marketing team are encouraged to proceed with a sequential reading of this document.

1.4 Product Scope

With the rise of interest in local exploration and cultural experiences, particularly amidst the challenges posed by the global pandemic, there has been a notable surge in demand for innovative approaches to discovering new locations.

GoWhere is a web application developed to cater to the evolving needs of adventure enthusiasts seeking to discover new locations effortlessly. It serves as a comprehensive platform designed to enhance the exploration experience by integrating real-time information and user-generated content. GoWhere enables users to navigate and explore diverse locations seamlessly, ensuring they have access to the latest updates on opening hours, weather, and reviews.

The application features a built-in map, assisting users in locating nearby points of interest and streamlining the decision-making process when planning their next adventure. Whether users are seeking hidden gems, popular landmarks, or off-the-beaten-path destinations, GoWhere provides curated suggestions tailored to their preferences and constraints.

As GoWhere becomes an integral part of the travel exploration ecosystem, it not only transforms the way people engage with new locations but also contributes to the promotion and preservation of small businesses. In a rapidly changing world, this innovative platform ensures that the richness and diversity of travel experiences remain accessible, celebrated, and enjoyed by all.

1.5 References

Documentation:

React.js <https://react.dev/learn>

Express.js <https://expressjs.com/>

Firebase. (2024, April 15). Firebase documentation. Firebase Docs.
<https://firebase.google.com/docs>

Google. (n.d.). Google.
<https://developers.google.com/maps/documentation/places/web-service/overview>

Mapbox
<https://docs.mapbox.com/mapbox-gl-js/api/>

Openweathermap <https://openweathermap.org/api>

2. Overall Description

2.1 Product Perspective

GoWhere is an innovative, new, and self-contained web application that aims to revolutionize the way people discover, access, and interact with locations in Singapore. It provides users with a new and random location suggestion based on the user's preference.

2.2 Product Functions

The system features of the GoWhere web application can be categorized into two primary subgroups: Accounts and Main. This segment offers a broad overview of the system features offered by the web application. Detailed information about each feature, including activity flows, conditions, assumptions, and accomplished functional requirements, is available in Section 4, titled "System Features."

2.2.1 Accounts:

1. Create Account
2. Account Login
3. View Account
4. Change Password
5. Forget Password
6. Change Profile Picture

2.2.2 Main:

1. The Application allows the User to state their preferences of a location through a 3-question questionnaire.
2. The Application allows the User to Randomly Search for a location based on their preferences.
3. The Application allows the User to view the location of the given location on the map.
4. The Application allows the User to view the location details of the given location.
5. The Application allows the User to save the given location.
6. The Application allows the User to view their saved locations.
7. The Application allows the User to unsave their saved locations.
8. The Application allows the User to view the current weather at the location.
9. The Application prompts the User to bring an Umbrella if it is currently raining at the given location.
10. The Application allows the User to view more reviews about the location.

2.3 User Classes and Characteristics

GoWhere expects its userbase to primarily be the following, ranked in order of their priority.

2.3.1 Locals (Singaporeans/PRs)

Attributes:	Description
Frequency of Use:	High
Functions Used:	1. Create Account 2. Account Login 3. Reset Password 4. View Account 5. Change Password 6. Random Search 7. View Map and Location Details 8. View More Reviews 9. View Current Weather 10. Save Locations 11. View Saved Locations
Technical Ability:	Medium
Characteristics:	Locals will use GoWhere to find a new location to visit based on their indicated preferences through the questionnaire.

2.3.2 Tourists

Attributes:	Description
Frequency of Use:	Medium
Functions Used:	1. Create Account 2. Account Login 3. View Account 4. Random Search 5. View Map and Location Details 6. View More Reviews 7. View Current Weather 8. Save Locations 9. View Saved Locations
Technical Ability:	Medium
Characteristics:	Tourists will use GoWhere to plan their vacation in Singapore and enjoy its vast variety of cultures.

2.4 Operating Environment

GoWhere is designed to operate on devices equipped with a web browser and is compatible with both Android and iOS platforms. GoWhere should support Android OS versions from 6.0 (Marshmallow) onwards, as well as iOS 10 and later versions.

To access all of GoWhere's features, the User must have an internet connection. The User can utilize both Wi-Fi and mobile data networks (3G, 4G, 5G).

GoWhere requires access to the User's device's location service, to provide the User with a random location according to the preferred distance they have selected.

GoWhere utilizes responsive design, enabling it to adapt to different screen sizes. This guarantees a uniform user experience across all devices, whilst ensuring its functionality remains consistent regardless of the device used.

2.4.1 Production Environment

Support	Description
HTML	• The User's internet browser must support at least HTML5 or above • HTML5 is the latest standard of HTML as of April 2024
CSS	• The 'User's internet browser must support at least CSS3 or above • CSS3 is the latest standard of CSS as of April 2024
JavaScript	• The User's internet browser must support JavaScript

2.4.2 Development Environment

Support	Description
React.js	• React is a JavaScript library for building user interfaces. Developers can use components through React to make their web applications dynamic and interactive • React provides smooth rendering of the UI by using a virtual DOM that only updates the necessary parts of the actual DOM when data is changed • React is known for its simplicity, reusability, and performance • The CodeCrafters used React.js to build all user interfaces of GoWhere as its reusable components make the development process much faster and cleaner • React.js v18.2.0 is used
Firebase	• Firebase provides a variety of tools and services which speed

	up the app development process whilst still improving the app's quality. Firebase offers features such as real-time database, authentication, cloud storage, and more. • Firebase's cloud firestore which we used, is a NoSQL database. It stores data in JSON format, which can be used through Firebase's JavaScript SDK. • Firebase provides built-in functionality for user authentication, allowing developers to easily integrate authentication features, such as account creation, login, and password reset, into their applications • Firebase v10.8.1 is used
--	---

2.5 Design and Implementation Constraints

Documents the constraints encountered by the CodeCrafters during the development and maintenance of GoWhere, as well as all its design standards.

2.5.1 Limitations

Covers the limitations which could affect some of GoWhere's functionalities.

Limitation	Description
Google Places API	Google Places offers a generous amount of functionality for free, however, the lack of sufficient data on prices across locations in Singapore hinders us from incorporating price as a factor for users when selecting their preferences.

2.5.2 Design Standards

Documents the design standards for GoWhere.

Conventions	Description
Programming	• Camel case is used to name non-class variables and functions • Pascal case is used to name class variables
User Interface	• Uses a responsive design to ensure uniform user experience across different devices with differing viewports

2.5.3 Regulatory Compliance

Web applications must adhere to data protection and privacy regulations, which include local and international standards such as GDPR (General Data Protection Regulation)

2.6 User Documentation

GoWhere is designed to be easy to understand and use, without the aid of a guide or instructions. The interface and features are tailored to be straightforward to navigate.

2.7 Assumptions and Dependence

2.7.1 APIs

2.7.1.1 Mapbox API

Dependency: GoWhere integrates Mapbox for mapping and geological services (eg. finding nearby locations within 2km radius)

Implication: Changes or disruptions to Mapbox may affect GoWhere's mapping features and the ability to locate and provide information about nearby locations.

2.7.1.2 2-Hour Weather Forecast Gov API

Dependency: GoWhere relies on 2-Hour Weather Forecast Gov API to obtain information about forecasted weather

Implication: Changes or disruptions to 2-Hour Weather Forecast Gov API may affect information regarding forecasted weather

2.7.1.3 Google Places API

Dependency: GoWhere relies on Google Places API to obtain information about the randomly generated locations, such as the address, opening hours, and user reviews

Implication: Changes or disruptions to Google Places API may affect the completeness and accuracy of the location's details

2.7.1.4 Openweathermap API

Dependency: GoWhere relies on Openweathermap API to obtain information about the current temperature

Implication: Changes or disruptions to Openweathermap API may affect the completeness and accuracy of the location's current temperature.

2.7.2 Firebase Database

Dependency: GoWhere relies on Firebase DB to store and retrieve user data

Implication: Changes or disruption to Firebase DB may affect GoWhere's account functionalities

2.7.3 Internet Connection

Assumption: Users must have a stable and reliable internet connection

Implication: Unreliable internet connectivity will give rise to inefficiency and problems when accessing GoWhere's features

2.7.4 Location Services

Assumption: Users must allow their location services to be used

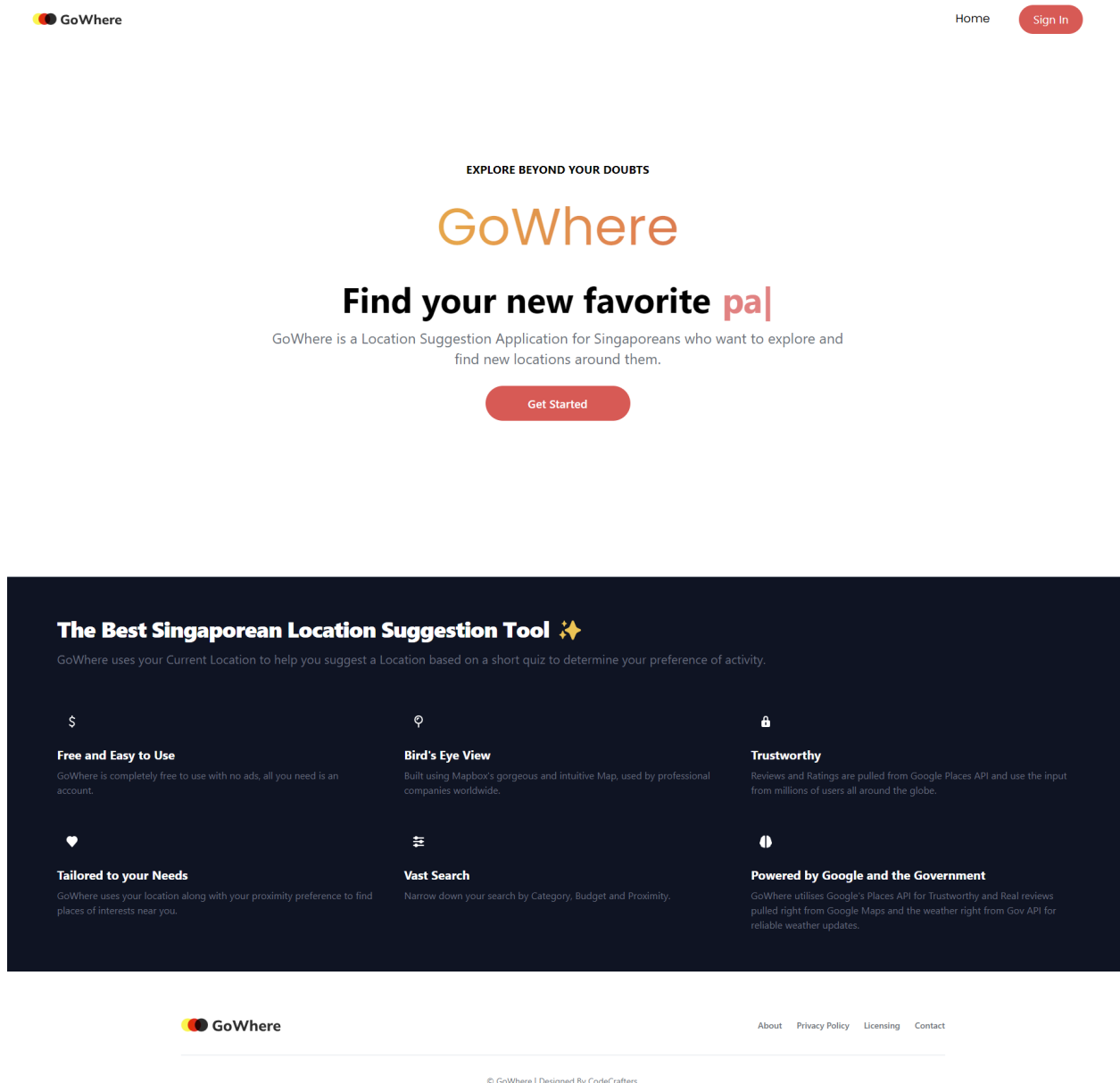
Implication: Without location services, it will give rise to inefficiency and problems when generating a suitable location for the User

3. External Interface Requirements

3.1 User Interfaces

GoWhere has a responsive design such that it is adaptable to different viewports on different devices. The internet browser used by the user must also satisfy the requirements as indicated within section 2.4.1 Production Environment of GoWhere.

3.1.1 Home Page



3.1.2 Create Account Page

Home [Sign In](#)

Create Account

Create an Account Today.

[Sign in](#)

[About](#) [Privacy Policy](#) [Licensing](#) [Contact](#)

© GoWhere | Designed By CodeCrafters.

3.1.3 Login Page



Home [Sign In](#)

Sign In

Welcome Back.

[Forgot Password?](#)
[Create Account](#)



[About](#) [Privacy Policy](#) [Licensing](#) [Contact](#)

© GoWhere | Designed By CodeCrafters.

3.1.4 Forget Password Page



Home [Sign In](#)

Forgot Password?

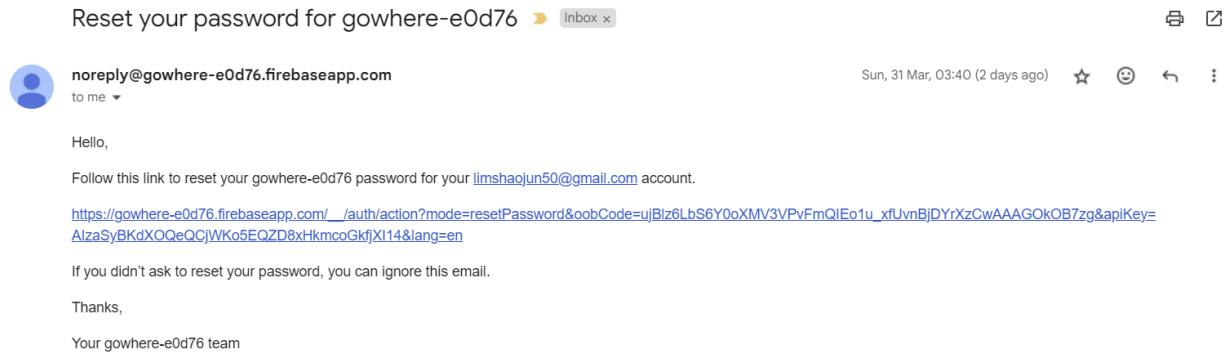
Don't worry, we can help you get back in.



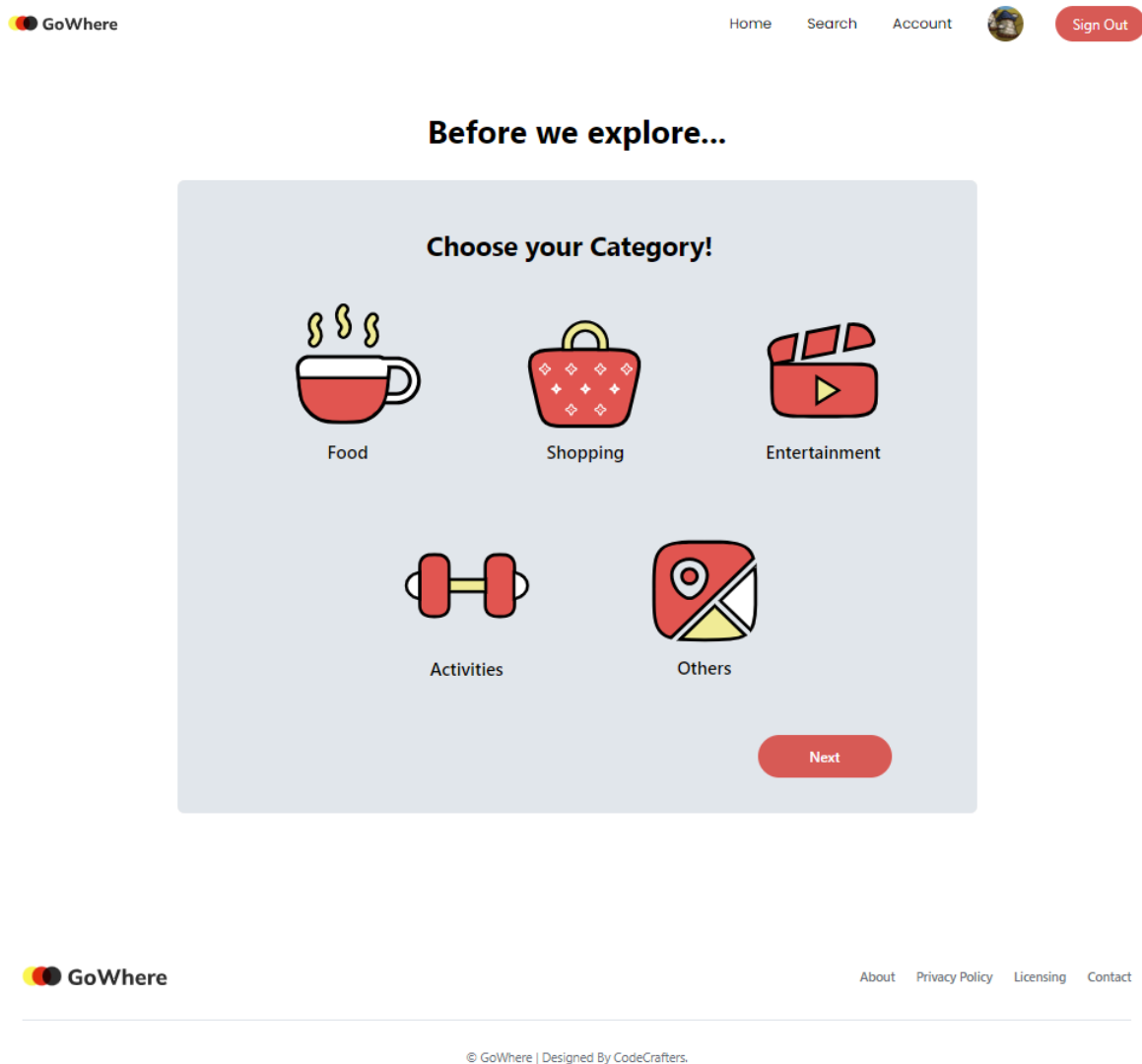
[About](#) [Privacy Policy](#) [Licensing](#) [Contact](#)

© GoWhere | Designed By CodeCrafters.

3.1.5 Email



3.1.6 Questionnaire Pages





Before we explore...

Do you want to exclude any Sub-categories?

Beauty Salon

Hair Care

Library

Spa

Tourist Attraction

[Back](#)

[Next](#)



Before we explore...

How far are you willing to go?

0 - 1 km

~ 10mins by Public Transport



0 - 5 km

~ 15mins by Public Transport



0 - 10 km

~ 20mins by Public Transport



0 - 20 km

~ 45mins by Public Transport



whole of Singapore

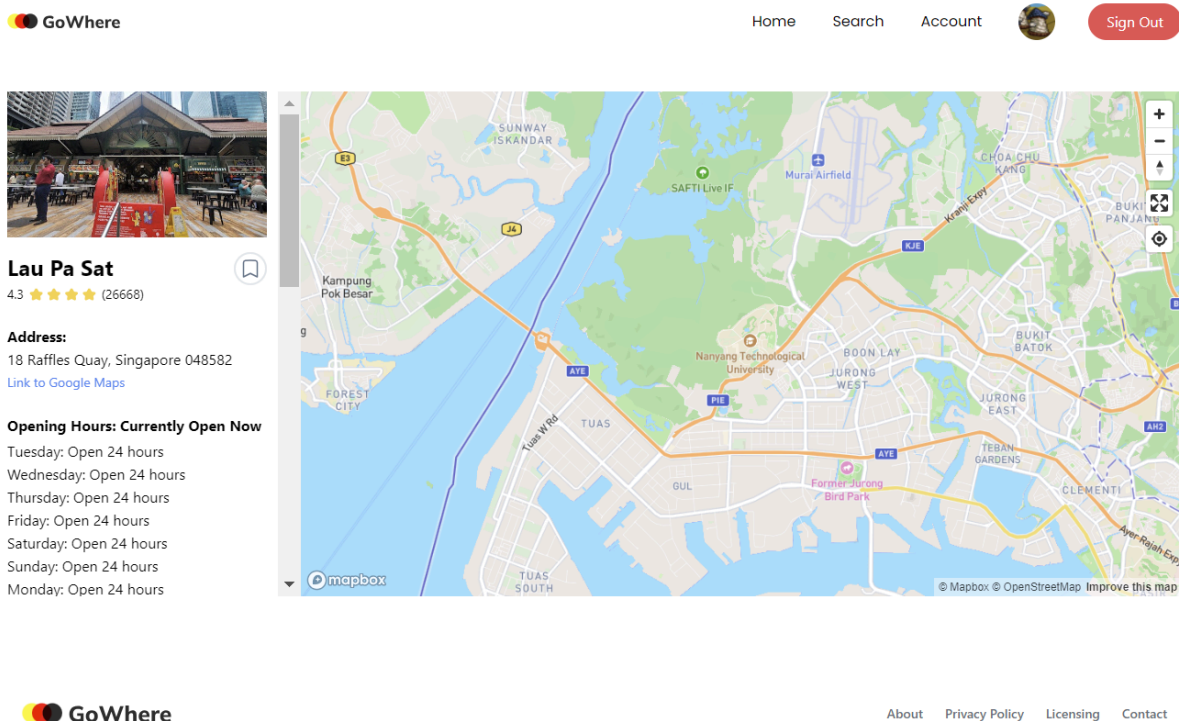
~ 1h 30mins from 1 end of Singapore to the other



[Back](#)

[Submit](#)

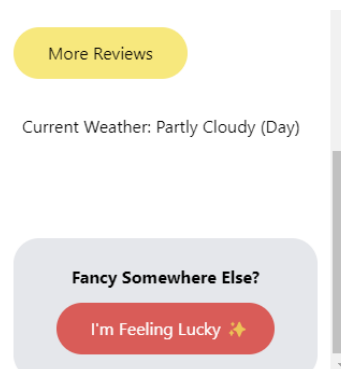
3.1.7 Map Page



Shows what is displayed after scrolling down on the left side

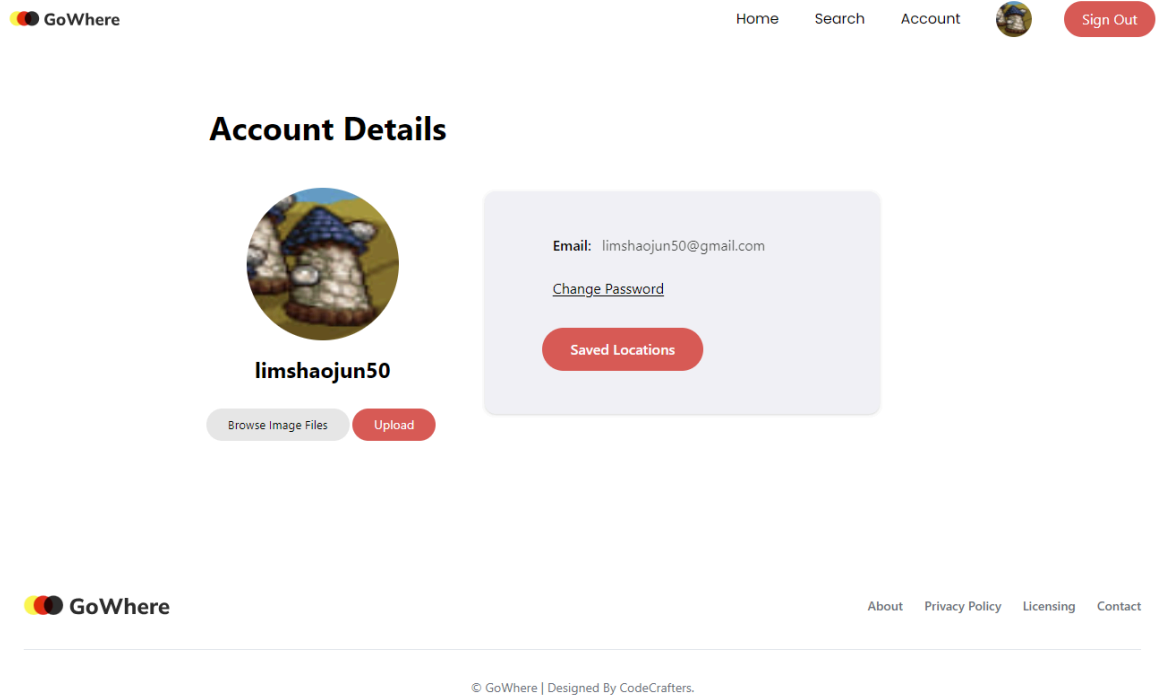


Part 1:



Part 2:

3.1.8 Account Details Page



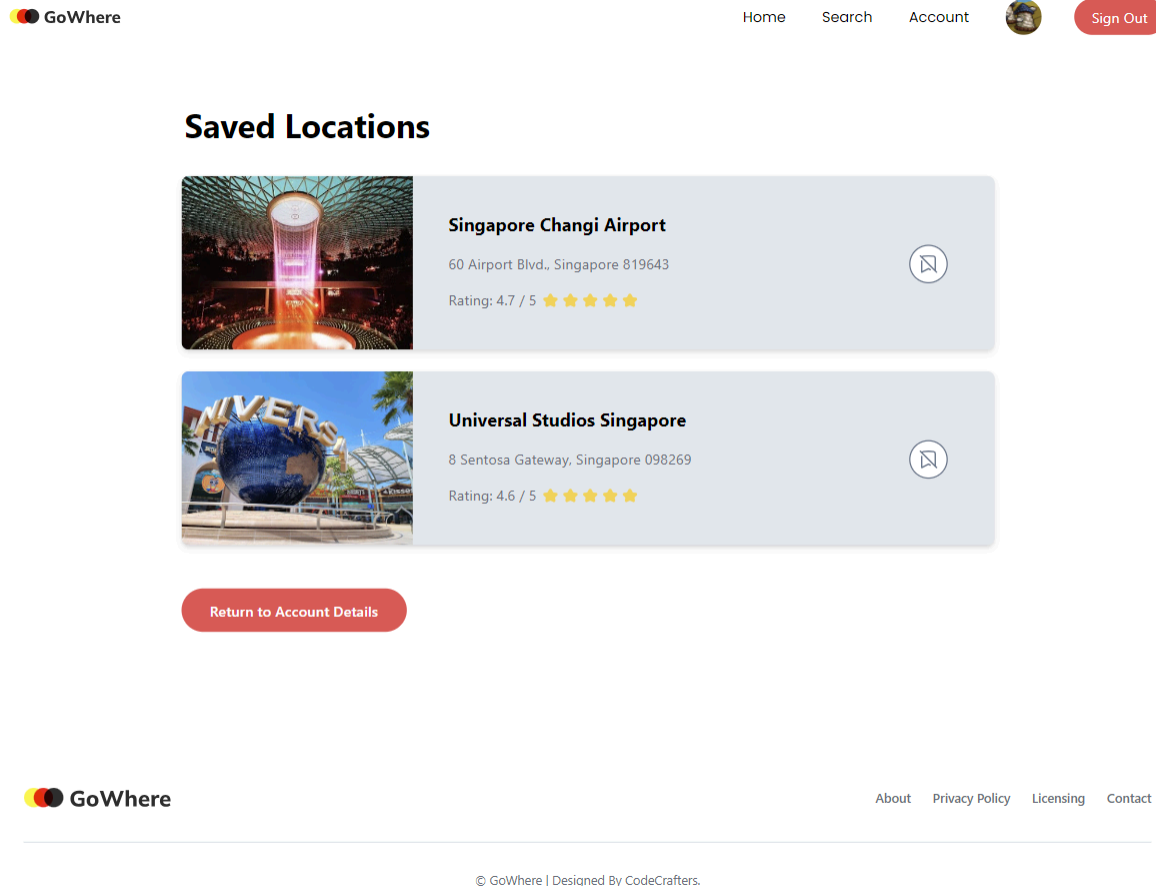
3.1.9 Change Password Page



Change Password

Change your password here.

3.1.10 Saved Locations Page



3.2 Hardware Interfaces

This section covers GoWhere's client-side and server-side hardware interface requirements.

3.2.1 Client-side Requirements

GoWhere can be used on both desktop and mobile platforms. The User's device must have a web browser that supports HTML5, CSS3 and JavaScript, as well as a stable internet connection to transmit and receive data from the server.

3.2.2 Server-side Requirements

The back-end server of GoWhere needs to be hosted and operated on a server computer. This server will handle Create, Read, Update, Delete (CRUD) operations on the back-end database. The database itself should be hosted and operated on a cloud-based server computer.

3.3 Software Interfaces

3.3.1 Software Components and Versions

The front-end of GoWhere is implemented with React.js, which is client-side rendering. This allows GoWhere to be more interactive whilst still running very smoothly. When a User loads a page, the user's browser interprets the HTML, CSS, and JavaScript received from the server and generates the DOM accordingly. Then, any updates to the DOM are handled by the user's browser in response to user interactions or JavaScript code execution. GoWhere also utilises the back-end server and database built using Firebase to handle storing of user data, and retrieval for authentication services.

3.4 Communications Interfaces

GoWhere's communication architecture follows a client-server model. It communicates with DataGovSG API, Firebase API, Google Places API, and Mapbox API using HTTPS protocol to ensure data security during transmission. Requests and responses will be in JSON format, following RESTful conventions. Before any request is made, all the APIs need to be initialised with an API key first to access the service securely.

2-Hour Weather Forecast Gov API is used to fetch the weather of the randomly generated location. Firebase API fetches data for the user profiles and saved locations' details. Mapbox API fetches the map and geolocation information. Google Places API fetches reviews, photos and opening hours.

4. System Features

The system will perform functionalities based on the Use Case Diagram and Sequence Diagram detailed in the Appendix.

4.1 Create Account

4.1.1 Description and Priority

Users can register for an account using their email and password at the Create Account Page.

Priority: High

4.1.2 Stimulus/Response Sequences

Flow of Events:	1. User clicks on the “Create Account” button to create an account. 2. The user will be redirected to the Account Creation page to register for a new account. 3. They will have to enter a valid email and password, where their email will be parsed and the part before the “@” will act as their username 4. They will also have to retype their password to confirm it in the confirm password input. 5. After the User clicks on “create account”, the User will be brought to the Sign In Page.
Alternative Flows:	AF-S1: User accidentally clicks on the “Sign in” button 1. The user will be redirected to the Sign In page to log into their account. (2.2 Account Login) 2. They will have the option to click the “Create Account” button to go back. AF-S3: User has an existing account with the provided email. 1. System prompts User to use a different email.

4.1.3 Functional Requirements

1. The User must be able to create a new account using the sign-up page.
 - 1.1. During registration, the User must provide an email and a password.
 - 1.1.1. The System must verify that the email provided is valid.
 - 1.1.1.1. The Email provided must have a string before and after the “@” Character.
 - 1.1.1.2. The System must verify if the email provided is already in the database
 - 1.1.1.2.1. If the email is already in the database, the System will prompt the user to enter another email.
 - 1.1.2. The System must verify that the password provided is valid.
 - 1.1.2.1 The Password provided must be of length greater than 5 and less than 30 characters long.
 - 1.1.2.2 The Password in “Password” and “Confirm Password” matches.
 - 1.2. To verify the user’s account details, the System must send a verification email to the user’s email
 - 1.2.1. The Email must contain a link which redirects them back to our website.
2. Upon successful creation, the User must be redirected to the questionnaire page.

4.2 Account Login

4.2.1 Description and Priority

Users can log in to their account using their registered email and password.

Priority: High

4.2.2 Stimulus/Response Sequences

Flow of Events:	1. User launches GoWhere Web Application. 2. User clicks on the "Sign In" button at the top right-hand corner and is redirected to the Sign in Page 3. User enters their email and password and clicks the "Sign In" button to login. 4. System verifies that the User has a valid account. 5. User is brought to the Account Details Page of the application.
Alternative Flows:	AF-S2: User clicks on the "Create Account" 1. The user will be redirected to the Account Creation page to register their account. (2.1 Create Account) 2. They will have the option to click "Go back" which returns them back to the homepage. AF-S2: User forgets their account password 3. User clicks on "Forget Password" 4. User will be redirected to the Forgot Password page (2.3 Reset Password) AF-S4: User enters a invalid email or password 5. User will be prompted to enter a valid email or password 6. Once a valid email and password is provided, the user will be brought to the homepage of the application.

4.2.3 Functional Requirements

1. The User must be able to login to the system using their Account.
 - 1.1. The User must provide their email and password to login
 - 1.2. The System must verify that the email and password provided are correct
 - 1.2.1. The Email and Password provided by the User must match the existing user details in the database
 - 1.2.2. If the email or password entered is invalid, an Error Message must be shown on screen to prompt the user to re-enter their email and password.
2. Upon successful login, the User must be redirected to the questionnaire page.

4.3 Reset Password

4.3.1 Description and Priority

Users would be able to reset their password if they forget their current password.

Priority: Medium

4.3.2 Stimulus/Response Sequences

Flow of Events:	1. User keys in their email. 2. User clicks on "Send Verification Link", which will be sent to their email. 3. Upon clicking the verification link sent to their email, the User is redirected to the reset password page. 4. After clicking "submit", the new password will be saved to the database. 5. User is redirected to the Account Login page to login.
Alternative Flows:	AF-S2: Email User provided does not exist in the database. 1. A pop up message will appear stating "Invalid email, please key in a valid email"

4.3.3 Functional Requirements

1. Users must key in their account's email.

1.1. System must verify the account corresponding to the email provided exists in the database.

1.1.1. If email does not exist in the database, System must display a pop up message stating "Invalid email, please key in a valid email".

2. To verify the user's account details, the System must send a verification email to the user's email

2.1. The Email must contain a link which redirects them back to our website.

3. Upon successful verification, the User must be redirected to the Reset Password page.

4. User must key in their new password.

4.1. The new password must be keyed in again in the confirm password section and must match the new password.

5. After the "submit" button is clicked, the old password must be replaced by the new password in the database.

4.4 View Account

4.4.1 Description and Priority

User can view the email and password associated with their account, and choose to change their password or view their saved locations.

Priority: Medium

4.4.2 Stimulus/Response Sequences

Flow of Events:	1. User clicks on the "Account" (depicted by their 2. Application loads in the User account page containing their email. 3. If the user clicks on change password, they will be redirected to the change password page. 4. If the user clicks on saved locations, they will be redirected to the saved locations page. 5. If the use clicks on the Browse Files button, they will be able to upload an image (either .jpg or png). 6. If the user clicks on the Upload Image, they will be able to then upload the chosen image as their profile photo.
Alternative Flows:	-

4.4.3 Functional Requirements

1. Users must be able to view their account.
 - 1.1. User Accounts must display their email.
 - 1.2. User Accounts must display a link to change their password.
 - 1.3. User Accounts must display a link to the list of the Users' Saved Locations
 - 1.4. User Accounts must allow Users to upload an image for their profile picture.

4.5 Change Password

4.5.1 Description and Priority

Users would be able to edit and change their account's password once logged in.

Priority: Low

4.5.2 Stimulus/Response Sequences

Flow of Events:	1. User keys in their old and new password. 2. User confirms their new password by keying it in one more time and clicks on the "Change Password" button 3. User's new password replaces the old password in the database
Alternative Flows:	AF-S2: The old and new passwords are identical 1. The System will prompt the User to key in a password that is different from the previous password. AF-S2: The new password and the confirm new password are different 1. The System will prompt the User that the 2 passwords are different and to key in identical passwords for those fields.

4.5.3 Functional Requirements

1. Users must key in their old and new passwords.

1.1. If the old and new passwords keyed in are identical, the System will display a pop-up message stating "Passwords are identical! Change denied"

1.1.1. The new password must be keyed in again in the confirm password section and must match the new password.

2. After the "submit" button is clicked, the old password must be replaced by the new password in the database.

4.6 Random Search

4.6.1 Description and Priority

Users will be posed a series of questions to help them get a location suggestion based on their preferences.

Priority: High

4.6.2 Stimulus/Response Sequences

Flow of Events:	The questions will be flashed one by one for a total of 3 questions, and questions are switched to the next question upon the completion of the current question. 1. 1st Question: User to pick 1 general category (Food, Shopping, Activities, Entertainment, Others). 2. 2nd Question: User is given a list of categories that have been narrowed down based on the general category they picked. 3. From this list, the User can exclude as many of the categories as they wish by ticking the category they want to exclude. 4. 3rd Question: User to allow System to use their Current Location and a list of ranges of proximity of locations from their location. 5. Upon submission of questionnaire, the User will be brought to the Map page (4.7 View Map and Location Details) where a random location that fulfills the above criteria will be generated for them along with the location details.
Alternative Flows:	AF-S1: User wants to change their input in a previous question(s). 1. User can go back to the previous questions using the “Go Back” button to cycle through their questions.
Exceptions	-

4.6.3 Functional Requirements

1. The System must provide a total of 3 questions to the User.
 - 1.1 The User must be logged in to access the questionnaire.
 - 1.2. The 1st Question must allow the User to pick one general category from 5 categories.
 - 1.2.1. The 5 categories are Food, Shopping, Entertainment, Activities, and Others.
 - 1.2.2. The User must select 1 general category before they can proceed to the next question.
 - 1.3. The 2nd Question must allow the User to pick any specific categories to exclude from the general category.
 - 1.3.1. The User does not have to tick any of the categories if they do not want to

exclude any categories.

1.3.2. The User must be able to pick multiple categories to exclude.

1.4. The 3rd Question must allow the User to pick from a list the range of distances they are willing to travel.

1.4.1. The User must select 1 range of distance before they can submit the questionnaire.

1.4.2. Before the user is able to submit this question, the System must display a pop-up message asking the user for permission to use their current location.

2. The user must click on the "Next" button to move on to the next question.

2.1. If the "Go Back" button is clicked instead, the page shall change to the previous question.

3. After filling in the questionnaire and clicking the "Submit" button, the System must change its page to the interactive map page. (next section 4.7)

3.1. If at least one location meets the criterias derived from the questionnaire, the location must be pinned and shown on the map page.

3.2. If no location which meets all criteria is found, the User will have to edit their location criteria through the questionnaire.

4.7 View Map and Location Details

4.7.1 Description and Priority

Users will see details of the randomized place they have been given, such as its name, ratings and reviews, address, current temperature and location on the map.

Priority: High

4.7.2 Stimulus/Response Sequences

Flow of Events:	1. Based on the questionnaire, Users are allocated a valid randomized location based on their preferences stated. 2. This location is then displayed on a Map page along with the location details in a side panel. 3. Users can click on the Google Maps link to find directions to the location 4. Users can click the bookmark icon to save this location in their account.
Alternative Flows:	AF-S1.1: User feels that the suggested location is not suitable and wants another location. 1. Users can click on the “I’m feeling lucky” button to be provided 2. Users will be provided with a new randomized location AF-S1.2: User changes their mind on the type of location wanted. 1. Users can click on the “Search” button in the Navigation Bar to be re-directed to the questionnaire. 2. Users can select their location criteria again.

4.7.3 Functional Requirements

1. The System must display an interactive map that shows the allocated random location.
 - 1.1. The Map must be able to grow and shrink around the User’s location.
 - 1.2. The User must have done the questionnaire before being re-directed to the Map.
2. The System must display the details of the allocated random location.
 - 2.1. These details include: Reviews, Ratings, Descriptions, and Address, Opening Hours, Current Weather and Temperature.
 - 2.1.1. System must pull these details through the Google Places API, 2-Hour Weather Forecast Gov API and Openweathermap API using the coordinates of the location.

4.8 View More Reviews

4.8.1 Description and Priority

Users can view more reviews about a location if they want to know more reviews about the location.

Priority: Low

4.8.2 Stimulus/Response Sequences

Flow of Events:	1. At the Map Page, User clicks on the “Read More Reviews” button. 2. System will call the Google Places API and pull 4 more reviews for the specified location. 3. System will then display the additional 4 reviews below the first review for a total of 5 reviews.
Alternative Flows:	-

4.8.3 Functional Requirements:

1. Users must be able to view more reviews of a specific location.

1.1. These details include: Reviews, Ratings and the Reviewer’s Name.

1.1.1. System must pull the details using the Google Places API using the name of the location.

4.9 View Current Weather

4.9.1 Description and Priority

User can view the current weather at the given location. Users would be prompted to bring an umbrella if the weather forecast predicts rainy weather.

Priority: Low

4.9.2 Stimulus/Response Sequences

Flow of Events:	1. Using the location or neighborhood of the given location, System retrieves the weather using the 2-Hour Weather Forecast Gov API and current temperature using the Openweathermap API. 2. The System displays the current weather and temperature of the location. 3. If the system receives information that there is going to be or is raining, the system will remind the User to bring an Umbrella.
Alternative Flows:	-

4.9.3 Functional Requirements

1. Whenever the user gets allocated a random location, the System must check the weather for the next 2 hours using the 2-Hour Weather Forecast Gov API and the current temperature using the Openweathermap API.

1.1. If the weather forecast predicts that it will rain within the next 2 hours, the System will display a message stating "It might rain, please bring an umbrella!" under the location details.

1.2. If the weather forecast does not predict any rain within the next 2 hours, the System will not show the warning message.

4.10 Save Locations

4.10.1 Description and Priority

Users would be able to save any location they want to revisit in the future and remove saved locations.

Priority: Medium

4.10.2 Stimulus/Response Sequences

Flow of Events:	1. In the Location Details on the Map Page, User clicks on the “Save Location” button (depicted by a white bookmark icon). 2. System adds the saved location to the User’s list of saved locations. 3. “Save Location” button changes to “Unsave Location” (Depicted by a gray bookmark icon).
Alternative Flows:	AF-S2: The User has already saved the location to their saved list. 1. The “Save Location” button has been changed to “Unsave Location” (Gray Bookmark Icon) 2. The User clicks on the “Unsave Location” button. 3. The System removes the location from the User’s saved locations list. 4. “Save Location” button changes back to the “Unsave Location”.

4.10.3 Functional Requirements:

1. Users must be able to save a location to their list of saved locations.
 - 1.1. The User must be logged into the Application before they can save a location.
2. If the location has already been saved before, Users must be able to unsave the location.
 - 2.1. The User must be logged into the Application before they can unsave a location.
 - 2.2. The System must change the “Save” option to the “Unsave” option.

4.11 View Saved Locations

4.11.1 Description and Priority

Users would be able to view previously saved locations.

Priority: Low

4.11.2 Stimulus/Response Sequences

Flow of Events:	1. Under the Account Details Page, User clicks on “View saved locations” 2. Users will be redirected to a page where they can view the list of their saved locations. Each location will have an “Unsave Location” button beside it. 3. Each location should display its details in an information card (Name, Address, Ratings).
Alternative Flows:	AF-S3: The User wants to Unsave a location. 1. The User clicks on the “Unsave Location” button and the saved location is removed from the list.

4.11.3 Functional Requirements:

1. Users must be able to view their list of saved locations.
 - 1.1. Users must be logged in and re-directed from the View Profile page to view this page.
 - 1.2. Each saved location should display its Name, Address, and Rating.
2. Users must be able to remove saved locations from their list of saved locations.

5. Other Non-functional Requirements

Non-Functional Requirements describe the properties the system must have, that is not directly related to the functional behavior of the system.

5.1 Performance Requirements

1. The landing page must provide an 8-second or less response time in a desktop browser.
2. Each search result must be displayed within 10 seconds.
3. The application must return the correct data to the user based on the selected value of inputs.
4. The web application must be functional on the latest versions of any browser type and in the operating system MacOS, Microsoft Windows, Android and iOS.

5.2 Security Requirements

1. The system must accept only a password with a minimum of 6 characters.
2. The system must ensure a user enters a valid email address and password to log into their account.
3. Users must be logged in using their registered email before using the search functions of the application.

5.3 Availability

1. The web application must maintain at least a 98% daily uptime for users located in Singapore, under normal network conditions.

5.4 Localization

1. The date in the system should follow Singapore Standard Time (UTC+8:00).
2. The default language of the web application should be English.

5.5 Usability

1. The system must return relevant results with an accuracy of at least 95% when using a search feature. Relevance is determined based on the distance, rating, and price chosen by the user.

5.6 Reliability

1. The system must not exceed an error rate of 5% for the completion of common tasks (eg. selecting customized inputs used to randomly search restaurants).

5.7 Maintainability

1. The system must maintain a mean time between failures (MTBF) of at least 30 days.
2. The system must have a maximum mean time to repair (MTTR) of 3 hours for critical failures.

5.8 Testability

1. The system must be capable of executing a comprehensive set of automated tests covering at least 90% of the codebase within an hour following any modifications made to the codebase.

5.9 Reusability

1. At least 60% percent of the entire codebase shall be reusable in the future projects requiring the same API.

7. Use Case Model



Figure 1: Use-Case Diagram

Use Case Descriptions

Use Case ID:	1		
Use Case Name:	Create Account		
Created By:	Shaojun	Last Updated By:	Keith
Date Created:	28/1/2024	Date Last Updated:	8/4/2024

Actor:	• User • Database
Description:	Users can register for an account using their email and password.
Preconditions:	• User must provide a valid email and password to create an account. • User must not have a GoWhere account with the email already. • The password provided must be of length greater than 5 and less than 30 characters.
Postconditions:	• User account created and stored in database • Re-directed to Login page
Priority:	High
Frequency of Use:	Med
Flow of Events:	1. User clicks on the "Create Account" button to create an account. 2. The user will be redirected to the Account Creation page to register for a new account. 3. They will have to enter their email and password, where their email will be parsed and the part before the "@" will act as their username 4. They will also have to retype their password to confirm it in the confirm password input. 5. After they click "Create Account", the User will be brought to the Login Page.
Alternative Flows:	AF-S1: User clicks on the "Sign in" button 1. The user will be redirected to the Sign In page to log into their account. (2.2 Account Login) 2. They will have the option to click "Go back" which returns them to the homepage. AF-S3: The User has an existing account with the email they provided

	1. System prompts User to use a different email.
Exceptions:	NIL
Includes:	NIL
Extends:	NIL
Special Requirements:	NIL
Assumptions:	NIL
Notes and Issues:	NIL

Use Case ID:	2		
Use Case Name:	Account Login		
Created By:	Shaojun	Last Updated By:	Keith
Date Created:	28/1/2024	Date Last Updated:	8/4/2024

Actor:	• User • Database
Description:	Users can login to their account using their registered email and password.
Preconditions:	• User has a GoWhere account
Postconditions:	• User is directed to the questionnaire page
Priority:	High
Frequency of Use:	High
Flow of Events:	1. User launches GoWhere Web Application. 2. User clicks on the “Sign In” button at the top right-hand corner and is redirected to the Sign in Page 3. User enters their email and password and clicks the “Sign In” button to login. 4. System verifies that the User has a valid account. 5. User is brought to the Account Details Page of the application.
Alternative Flows:	AF-S2: User clicks on the “Create Account” 1. The user will be redirected to the Account Creation page to register their account. (2.1 Create Account) 2. They will have the option to click “Go back” which returns them back to the homepage. AF-S2: User forgets their account password 1. User clicks on “Forget Password” 2. User will be redirected to the Forgot Password page. AF-S4: User enters a invalid email or password 1. User will be prompted to enter a valid email or password 2. Once a valid email and password is provided, the user will be brought to the homepage of the application.
Exceptions:	NIL
Includes:	NIL
Extends:	Reset Password
Special Requirements:	Email and Password are compulsory
Assumptions:	NIL
Notes and Issues:	NIL

Use Case ID:	3		
Use Case Name:	Reset Password		
Created By:	Shaojun	Last Updated By:	Keith
Date Created:	17/2/2024	Date Last Updated:	8/4/2024

Actor:	• User • Database
Description:	Users would be able to reset their password if they forget their current password.
Preconditions:	• User has a GoWhere account • User has forgotten their password
Postconditions:	• User's password has been reset • User is redirected to the Login Page
Priority:	Medium
Frequency of Use:	Medium
Flow of Events:	1. User keys in their email. 2. User clicks on "Send Verification Link", which will be sent to their email. 3. Upon clicking the verification link sent to their email, the User is redirected to the reset password page. 4. After clicking "submit", new password will be saved to the database.
Alternative Flows:	AF-S2: Email User provided does not exist in the database. 1. An error message will appear stating "Invalid email, please key in a valid email."
Exceptions:	NIL
Includes:	NIL
Special Requirements:	Email and Password are compulsory fields
Assumptions:	NIL
Notes and Issues:	NIL

Use Case ID:	4		
Use Case Name:	View Account		
Created By:	Shaojun	Last Updated By:	Shaojun
Date Created:	3/2/2024	Date Last Updated:	9/4/2024

Actor:	• User • Database
Description:	User can view the email associated with their account, and choose to change their password or view their saved locations or update their profile picture.
Preconditions:	• User has a GoWhere account • User is signed into their account
Postconditions:	• User can change their password • User can view their saved locations • User can change their profile picture
Priority:	High
Frequency of Use:	High
Flow of Events:	1. User clicks on the "Account" (depicted by their 2. Application loads in the User account page containing their email. 3. If the user clicks on change password, they will be redirected to the change password page. 4. If the user clicks on saved locations, they will be redirected to the saved locations page. 5. If the user clicks on the Upload Image, they will be able to change their profile picture by uploading an image.
Alternative Flows:	NIL
Exceptions:	NIL
Includes:	Account Login
Extends:	View Saved Locations, Change Password
Special Requirements:	NIL
Assumptions:	NIL
Notes and Issues:	NIL

Use Case ID:	5		
Use Case Name:	Change Password		
Created By:	Shaojun	Last Updated By:	Shaojun
Date Created:	3/2/2024	Date Last Updated:	17/2/2024

Actor:	• User • Database
Description:	Users would be able to edit and change their account's password.
Preconditions:	• User must be logged in • User must click on the change password option on the account details page.
Postconditions:	• User's password will be changed in the database • User will be redirected to the account details page
Priority:	Low
Frequency of Use:	Low
Flow of Events:	1. User keys in their old and new password. 2. User confirms their new password by keying it in one more time and clicks on the "Change Password" button 3. User's new password replaces the old password in the database 4. User is redirected back to the Account Details page.
Alternative Flows:	NIL
Exceptions:	NIL
Includes:	NIL
Extends:	NIL
Special Requirements:	NIL
Assumptions:	NIL
Notes and Issues:	NIL

Use Case ID:	6		
Use Case Name:	Random Search		
Created By:	Shaojun	Last Updated By:	Shaojun
Date Created:	28/1/2024	Date Last Updated:	8/4/2024

Actor:	User
Description:	Users will be posed a series of questions to help them get a location suggestion based on the preferences they provide.
Preconditions:	- User must be logged in 1st and 3rd Questions must be answered to proceed
Postconditions:	User will be directed to a map with details of a randomized location.
Priority:	High
Frequency of Use:	High
Flow of Events:	The questions will be flashed one by one for a total of 3 questions, and questions are switched to the next question upon the completion of the current question. 1st Question: User to pick 1 general category (Food, Shopping, Activities, Entertainment, Others). 2nd Question: User is given a list of categories that have been narrowed down based on the general category they picked. - From this list, the User can exclude as many of the categories as they wish by ticking the category they want to exclude. 3rd Question: User to allow System to use their Current Location and a list of proximity ranges of locations from their location. - Upon submission of the questionnaire, the User will be brought to the Map page (2.6 View Map and Location Details) where a random location that fulfills the above criteria will be generated for them along with the location details.
Alternative Flows:	AF-S1: User wants to change their input in a previous question(s). User can go back to the previous questions using the "Go Back" button to cycle through their questions.
Exceptions:	NIL
Includes:	Account Login, View Map and Location Details
Special Requirements:	NIL
Assumptions:	NIL
Notes and Issues:	NIL

Use Case ID:	7		
Use Case Name:	View Map and Location Details		
Created By:	Shaojun	Last Updated By:	Shaojun
Date Created:	3/2/2024	Date Last Updated:	10/4/2024

Actor:	• User • Google Places API • Mapbox API • Openweathermap API • 2-Hour Weather Forecast Gov API
Description:	Users will see details of the randomized place they have been given, such as its name, ratings and reviews, address, current temperature and location on the map.
Preconditions:	• The Location given must be within Singapore • A valid location must be generated by preferences indicated in the questionnaire to search it on the Mapbox Map.
Postconditions:	• User can choose to be directed to Google Maps for directions. • User can choose to save this location. • User is shown the location on the Map Page • User is shown the location details of the given location in a side bar • User is shown the current weather and temperature
Priority:	High
Frequency of Use:	High
Flow of Events:	1. Based on the questionnaire, Users are allocated a valid randomized location based on their preferences stated. 2. This location is then displayed on a Map page along with the location details in a sidebar. 3. Users can click on the Google Maps link to find directions to the location 4. Users can click on the bookmark icon to save this location in their account.
Alternative Flows:	AF-S1.1: User feels that the suggested location is not suitable and wants another location. 1. Users can click on the “I’m feeling lucky” button to be provided 2. Users will be provided with a new randomized location

	AF-S1.2: User changes their mind on the type of location wanted. 1. Users can click on the “Search” button in the Navigation Bar to be re-directed to the questionnaire. 2. Users can select their location criteria again.
Exceptions:	NIL
Includes:	NIL
Extends:	Save Locations, Weather Details, View More Reviews
Special Requirements:	NIL
Assumptions:	NIL
Notes and Issues:	NIL

Use Case ID:	8		
Use Case Name:	View More Reviews		
Created By:	Shaojun	Last Updated By:	Shaojun
Date Created:	17/2/2024	Date Last Updated:	10/4/2024

Actor:	• User • Google Places API
Description:	Users can view more reviews about a location if they want to know more reviews about the location.
Preconditions:	• User must have been given a randomised location. • The location must have more than 1 review • User is at the Map Page
Postconditions:	• User will shown a list of 5 reviews.
Priority:	Low
Frequency of Use:	Low
Flow of Events:	1. At the Map Page, User clicks on the "Read More Reviews" button. 2. System will call the Google Places API and pull 4 more reviews for the specified location. 3. System will then display the additional 4 reviews below the first review.
Alternative Flows:	NIL
Exceptions:	NIL
Includes:	NIL
Special Requirements:	NIL
Assumptions:	NIL
Notes and Issues:	NIL

Use Case ID:	9		
Use Case Name:	View Current Weather		
Created By:	Shaojun	Last Updated By:	Shaojun
Date Created:	3/2/2024	Date Last Updated:	8/4/2024

Actor:	• User • 2-Hour Weather Forecast Gov API • Openweathermap API
Description:	Users would be prompted to bring an umbrella if the weather forecast predicts rainy weather at the time the User is given their randomized location.
Preconditions:	• User has been given a randomised location on the map
Postconditions:	• User is shown the current weather and temperature • User is prompted to bring an umbrella if it is raining at the given location.
Priority:	Low
Frequency of Use:	High
Flow of Events:	1. Using the User's coordinates from their current location, System retrieves the current weather and temperature using the 2-Hour Weather Forecast Gov API 2. The System displays the current weather conditions of the location. 3. If the system receives information that there is going to be or is raining, the system will remind the User to bring an Umbrella.
Alternative Flows:	NIL
Exceptions:	NIL
Includes:	NIL
Special Requirements:	NIL
Assumptions:	NIL
Notes and Issues:	NIL

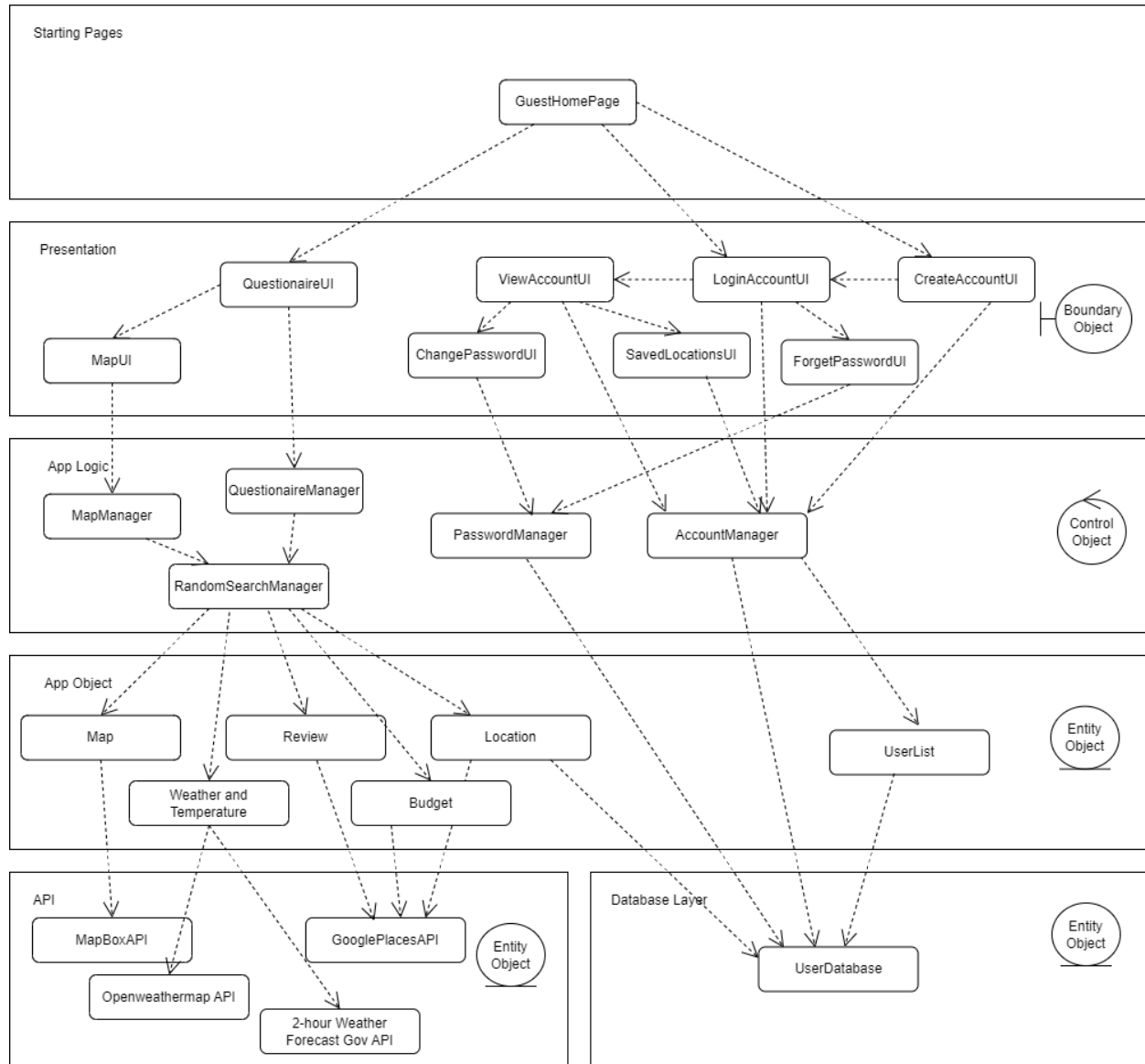
Use Case ID:	10		
Use Case Name:	Save Locations		
Created By:	Shaojun	Last Updated By:	Shaojun
Date Created:	3/2/2024	Date Last Updated:	3/4/2024

Actor:	• User • Database
Description:	Users would be able to save they want to revisit in the future and remove saved locations.
Preconditions:	• User must be signed into their account
Postconditions:	• Location details will be added to Saved Locations page
Priority:	Medium
Frequency of Use:	Medium
Flow of Events:	1. In the Location Details on the Map Page, User clicks on the “Save Location” button (depicted by a white bookmark icon). 2. System adds the saved location to the User’s list of saved locations. 3. “Save Location” button changes to “Unsave Location” (Depicted by a gray bookmark icon).
Alternative Flows:	AF-S2: The User has already saved the location to their saved list. 1. The “Save Location” button has been changed to “Unsave Location” (Gray Bookmark Icon) 2. The User clicks on the “Unsave Location” button. 3. The System removes the location from the User’s saved locations list. 4. “Save Location” button changes back to the “Unsave Location”.
Exceptions:	NIL
Includes:	NIL
Special Requirements:	NIL
Assumptions:	NIL
Notes and Issues:	NIL

Use Case ID:	11		
Use Case Name:	View Saved Locations		
Created By:	Shaojun	Last Updated By:	Shaojun
Date Created:	3/2/2024	Date Last Updated:	3/4/2024

Actor:	• User • Database
Description:	Users would be able to view previously saved locations.
Preconditions:	• User must be signed into their account
Postconditions:	NIL
Priority:	Low
Frequency of Use:	Medium
Flow of Events:	1. Under the Account Details Page, User clicks on “View saved locations” 2. Users will be redirected to a page where they can view the list of their saved locations. Each location will have an “Unsave Location” button beside it. 3. Each location should display its details in an information card (Name, Address, Ratings).
Alternative Flows:	AF-S3: The User wants to Unsave a location. 1. The User clicks on the “Unsave Location” button and the saved location is removed from the list.
Exceptions:	NIL
Includes:	NIL
Extends:	NIL
Special Requirements:	NIL
Assumptions:	NIL
Notes and Issues:	NIL

8. System Architecture - Lab 3 Deliverables



9. Other Requirements

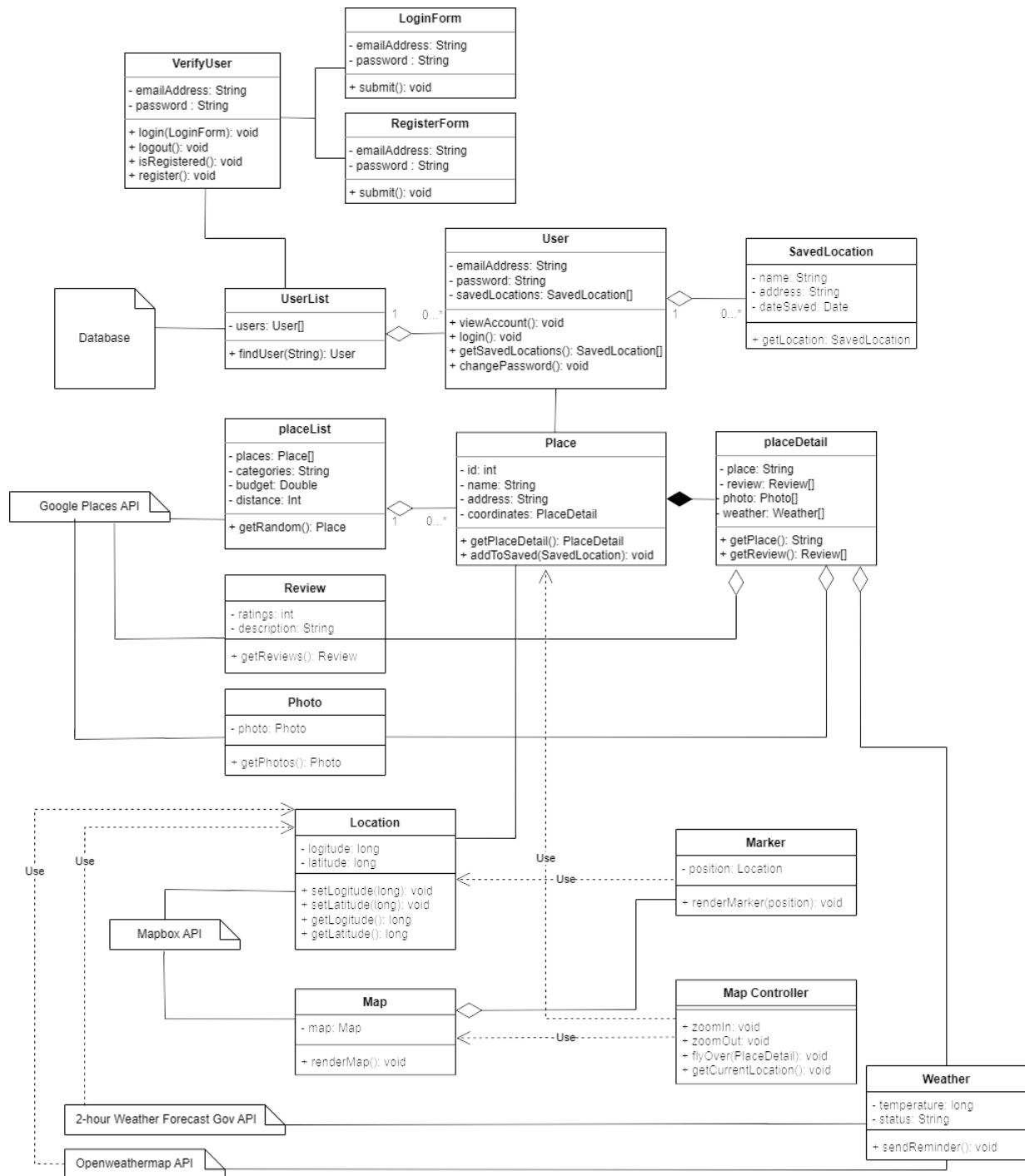
Appendix A: Glossary

9.1 Data Dictionary

Term	Definition
Places of Interest	The places that the user often frequents.
Category	The group of places that serve similar purposes or services. E.g. restaurants, supermarkets, clothing stores, etc... Type.
Area	The circular area around the chosen location that the user is willing to travel to.
Rating	The rating of the place provided by users of google maps
Budget	The amount of money that the user is willing to spend.
User	The person who is currently using the application.
Stakeholders	The organizations or people who have a vested interest in the application.
System	The application that is being used.
Login System	The part of the application that focuses on logging in.
Food Places	Places that serve food. Eateries such as Restaurants, Cafes, Hawker Centres, Food Courts, Canteens
Requirements	The baseline necessities to be met by the application.
Homepage	The page of the application that is first viewed by the user upon logging in.

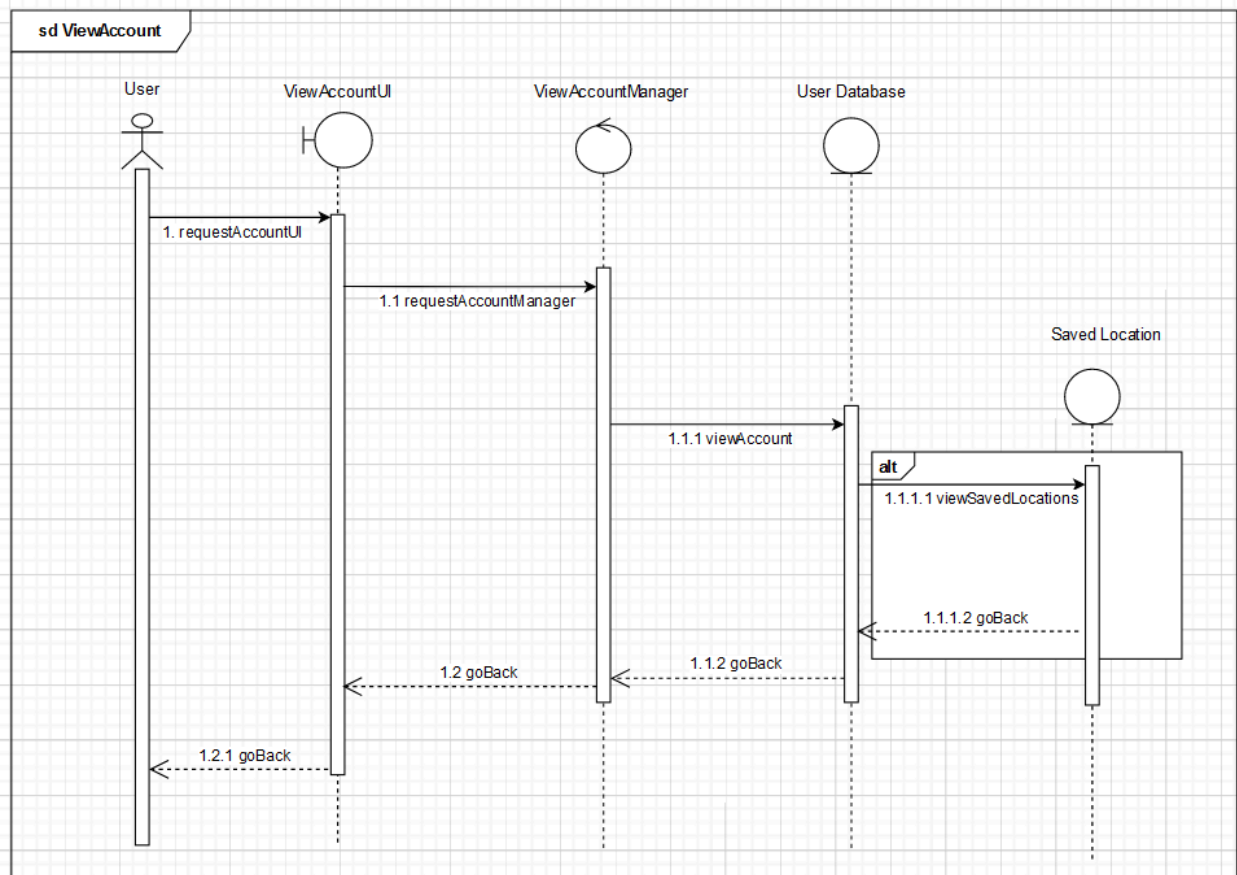
Appendix B: Analysis Models

9.2 Class Diagram (Conceptual Model)

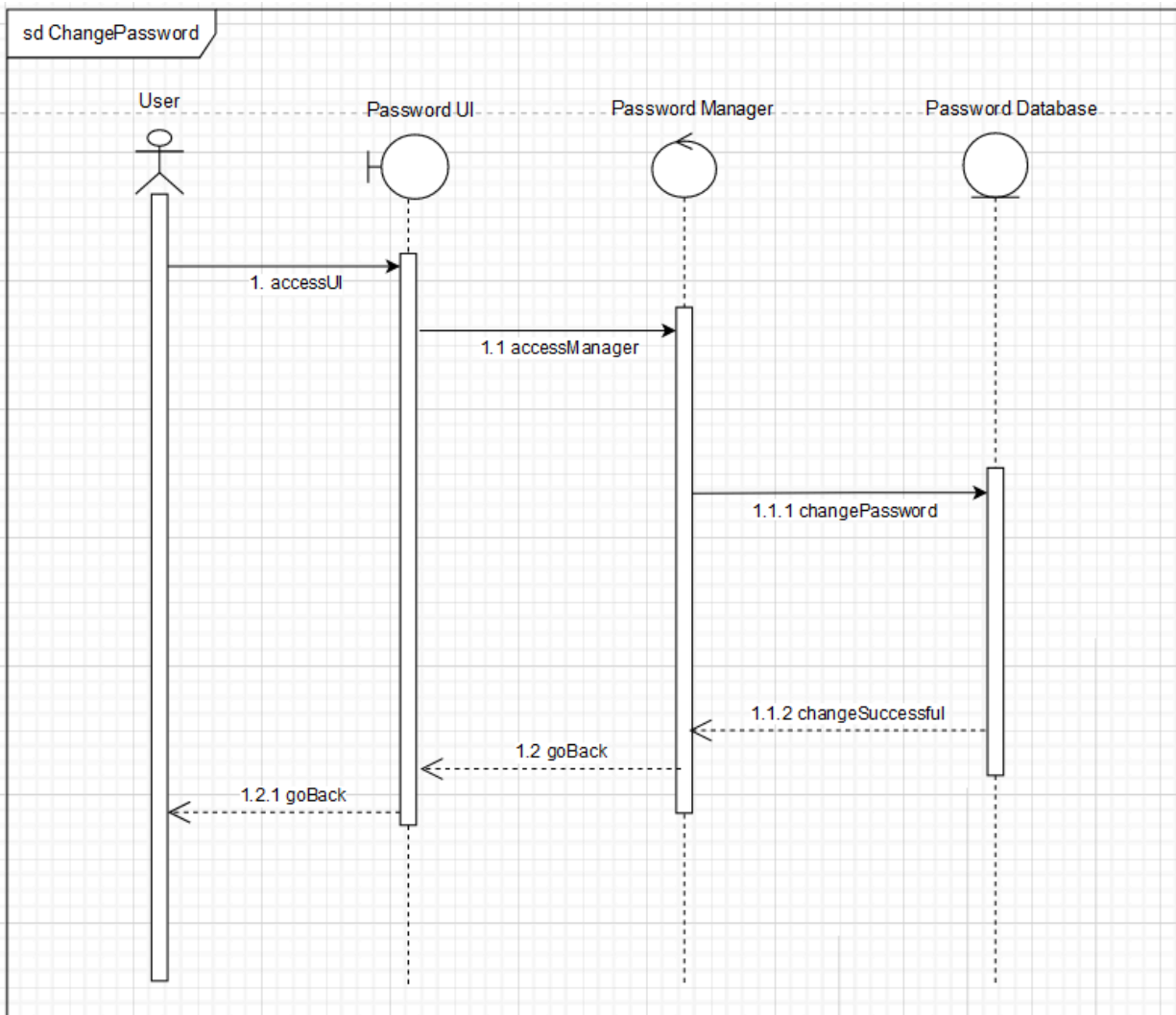


9.3 Sequence Diagram (Dynamic Model)

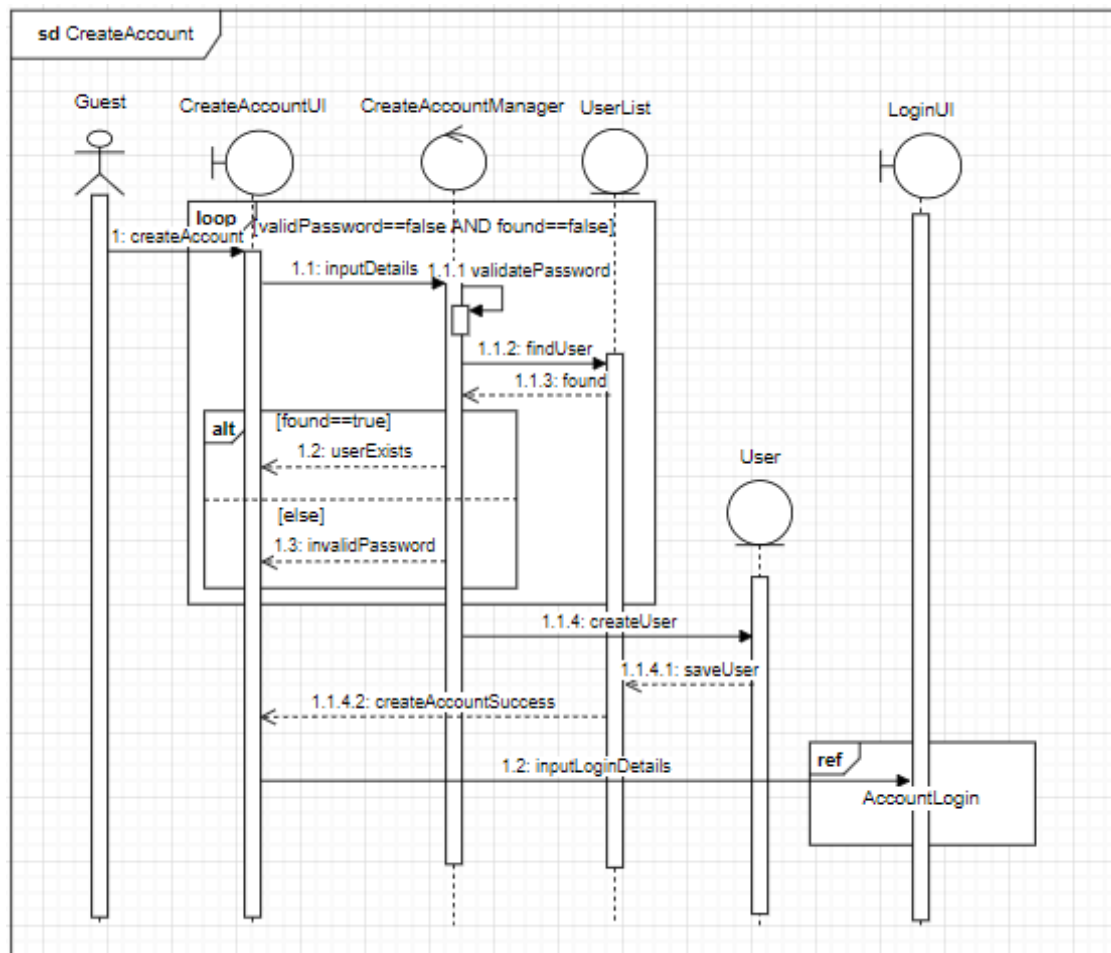
9.3.1 View account/View saved locations



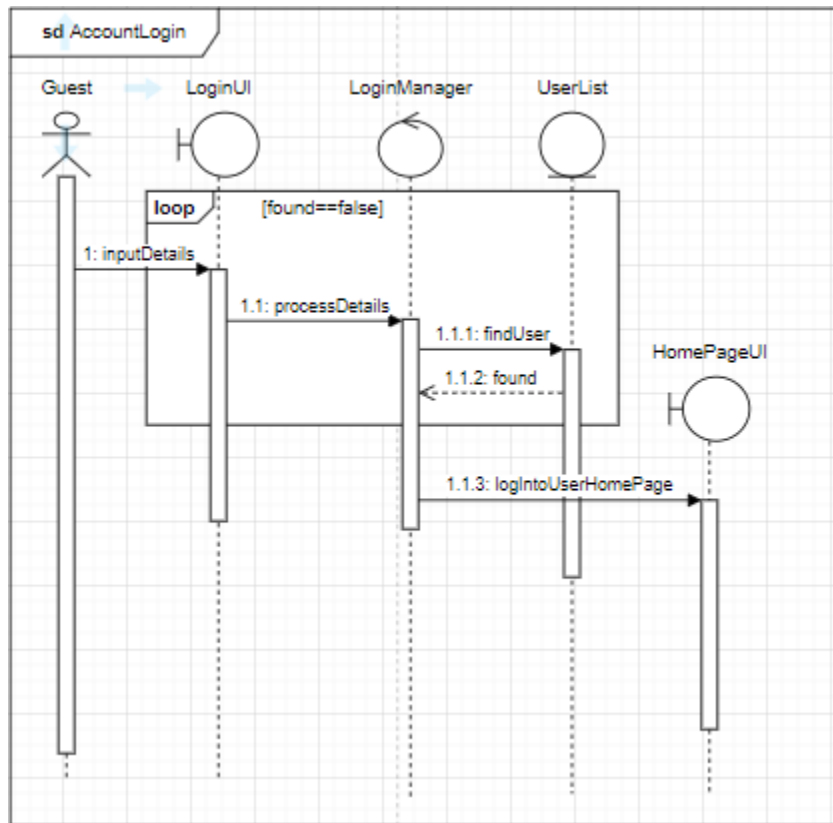
9.3.2 Change password



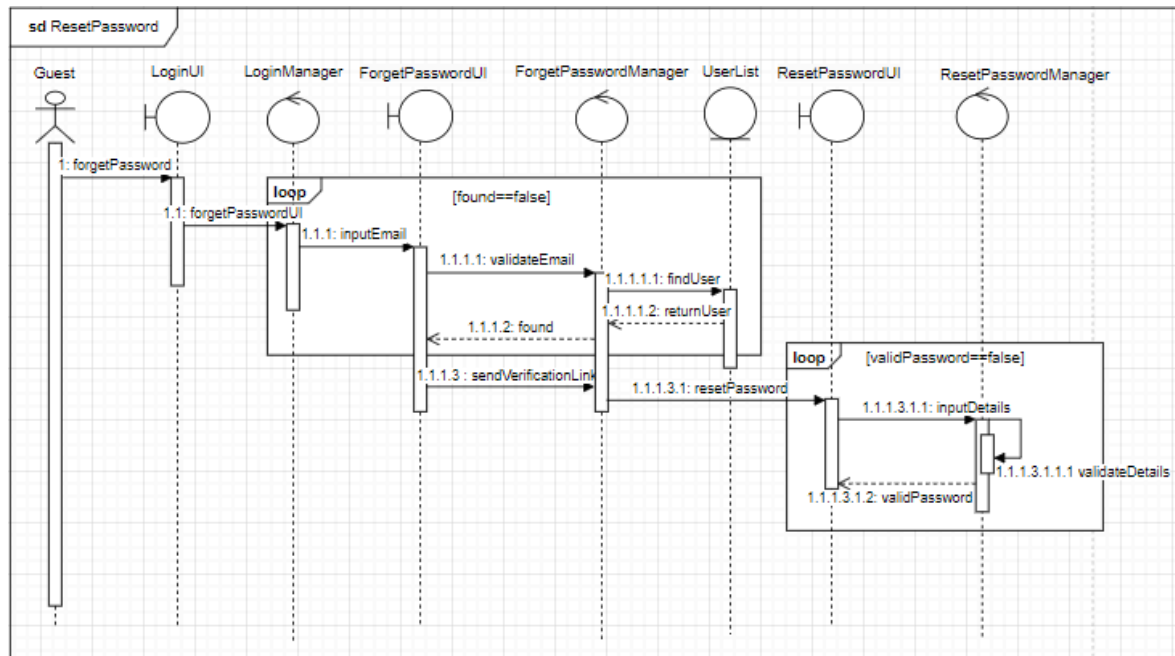
9.3.3 Create account



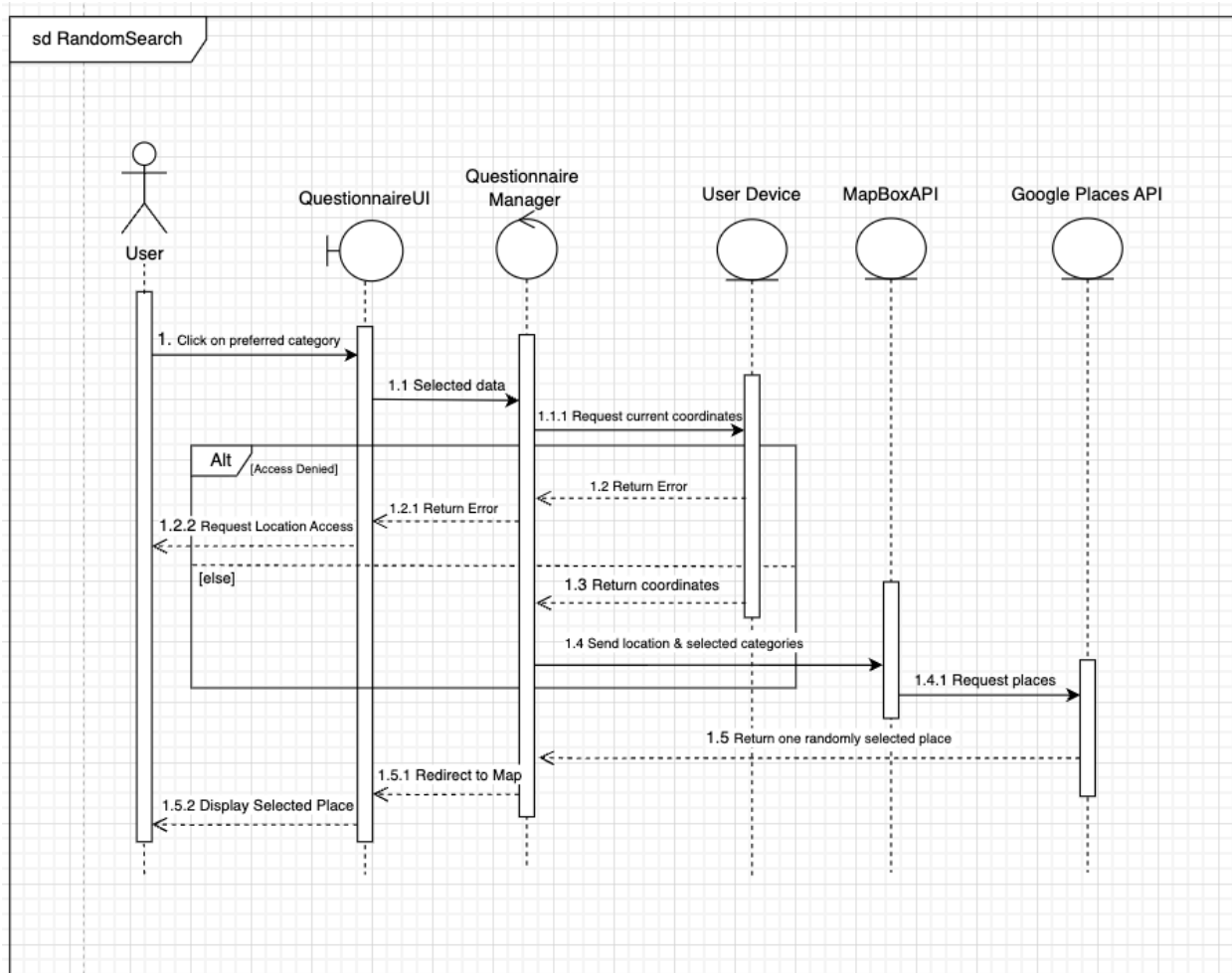
9.3.4 Account login



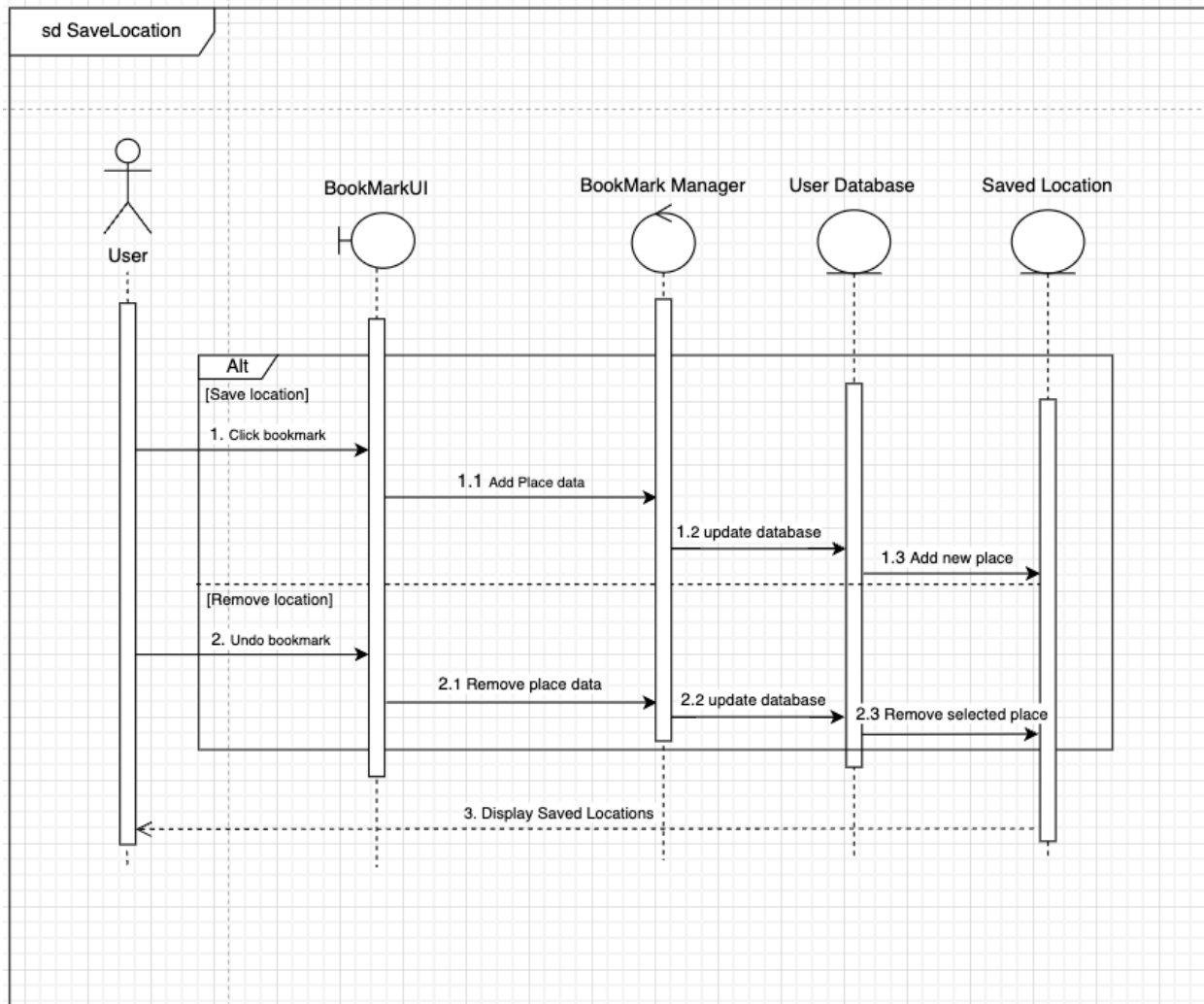
9.3.5 Reset password



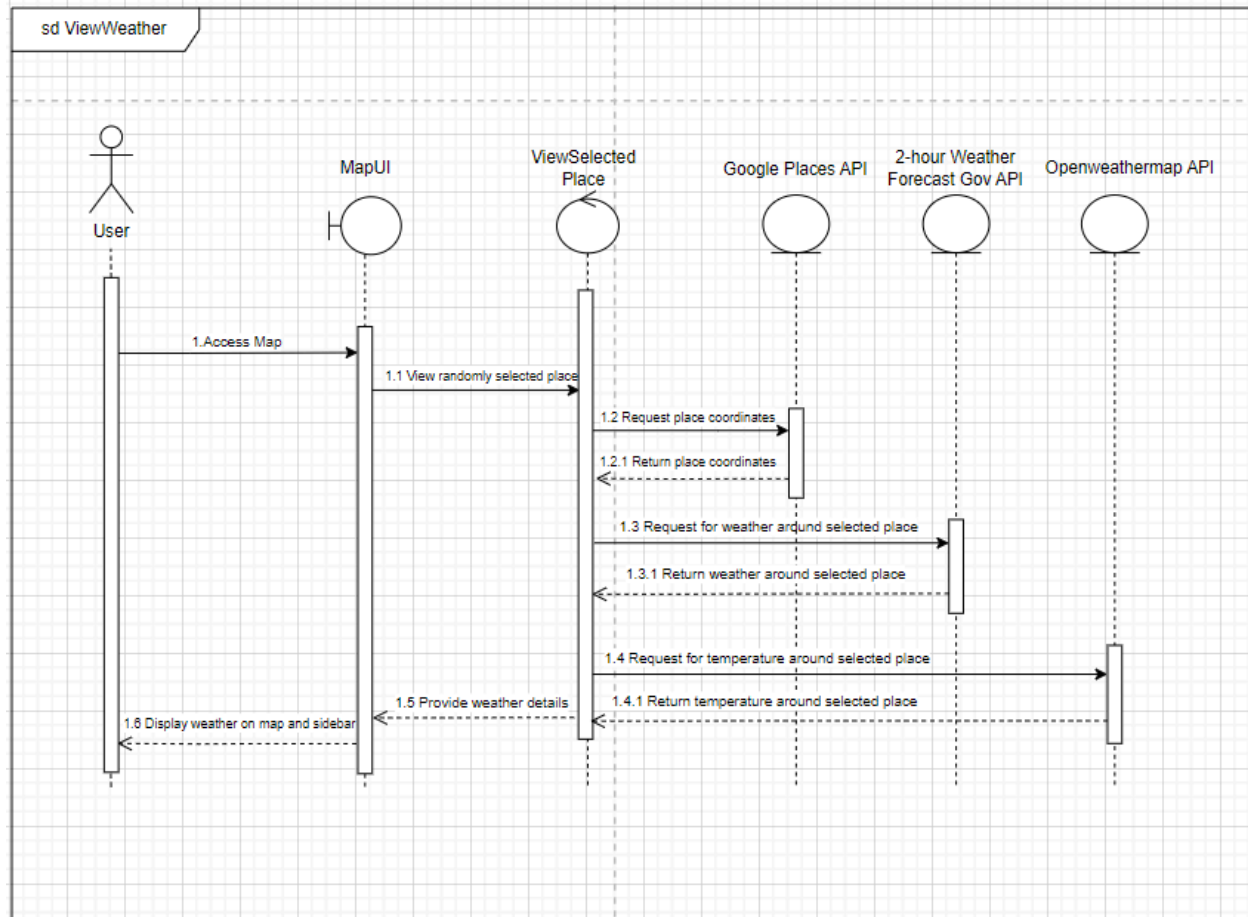
9.3.6 Random Search



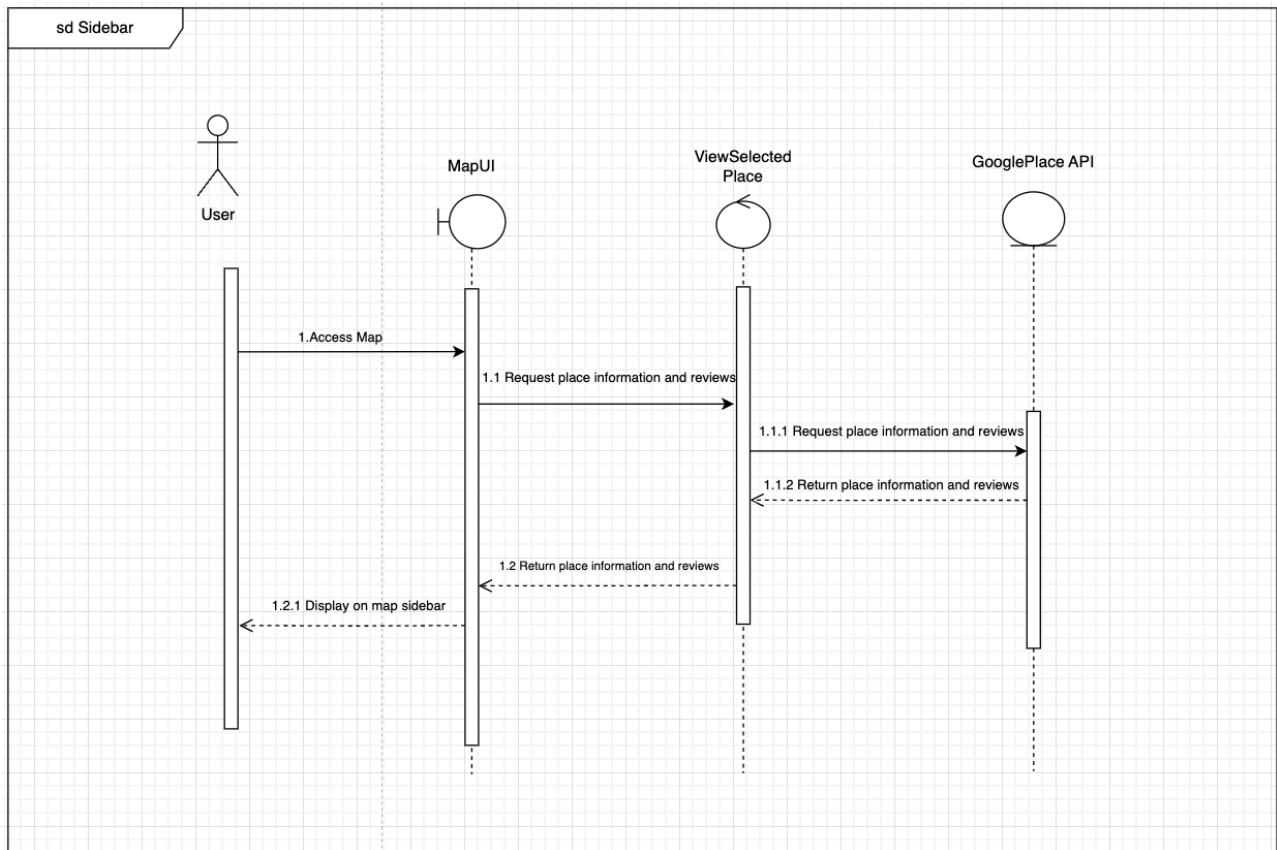
9.3.7 Save Locations



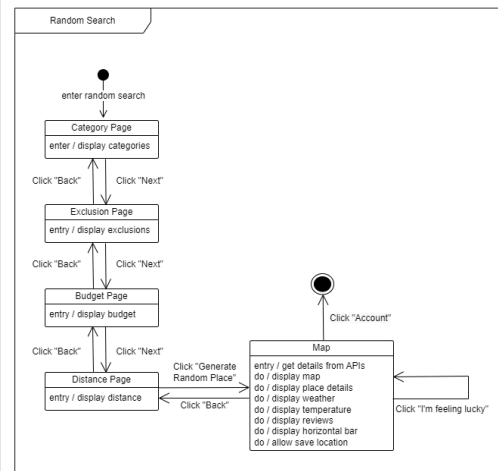
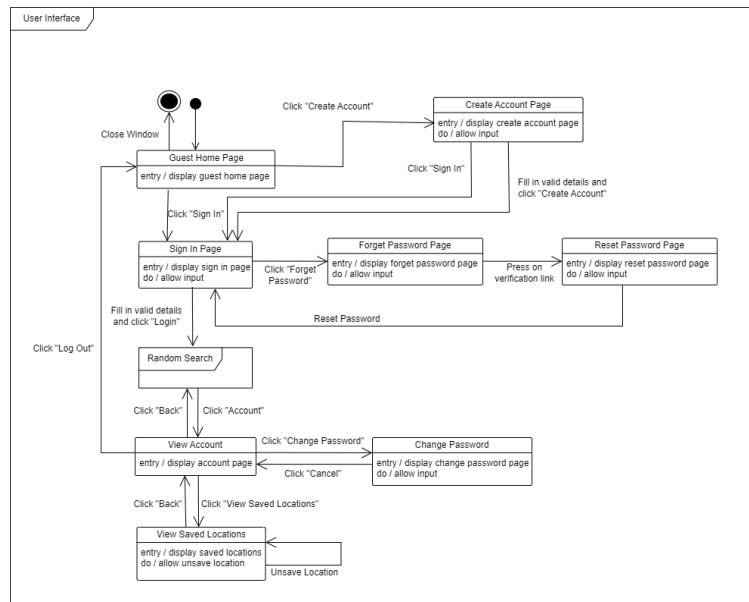
9.3.8 View Weather Information



9.3.9 View Place Information on Sidebar



9.4 Dialog Map (Dynamic Model)



9.5 Test cases and results

Black box testing

Since there is no boundary value testing available for any of our functions, we will only be conducting an equivalence class testing.

Create account

Inputs

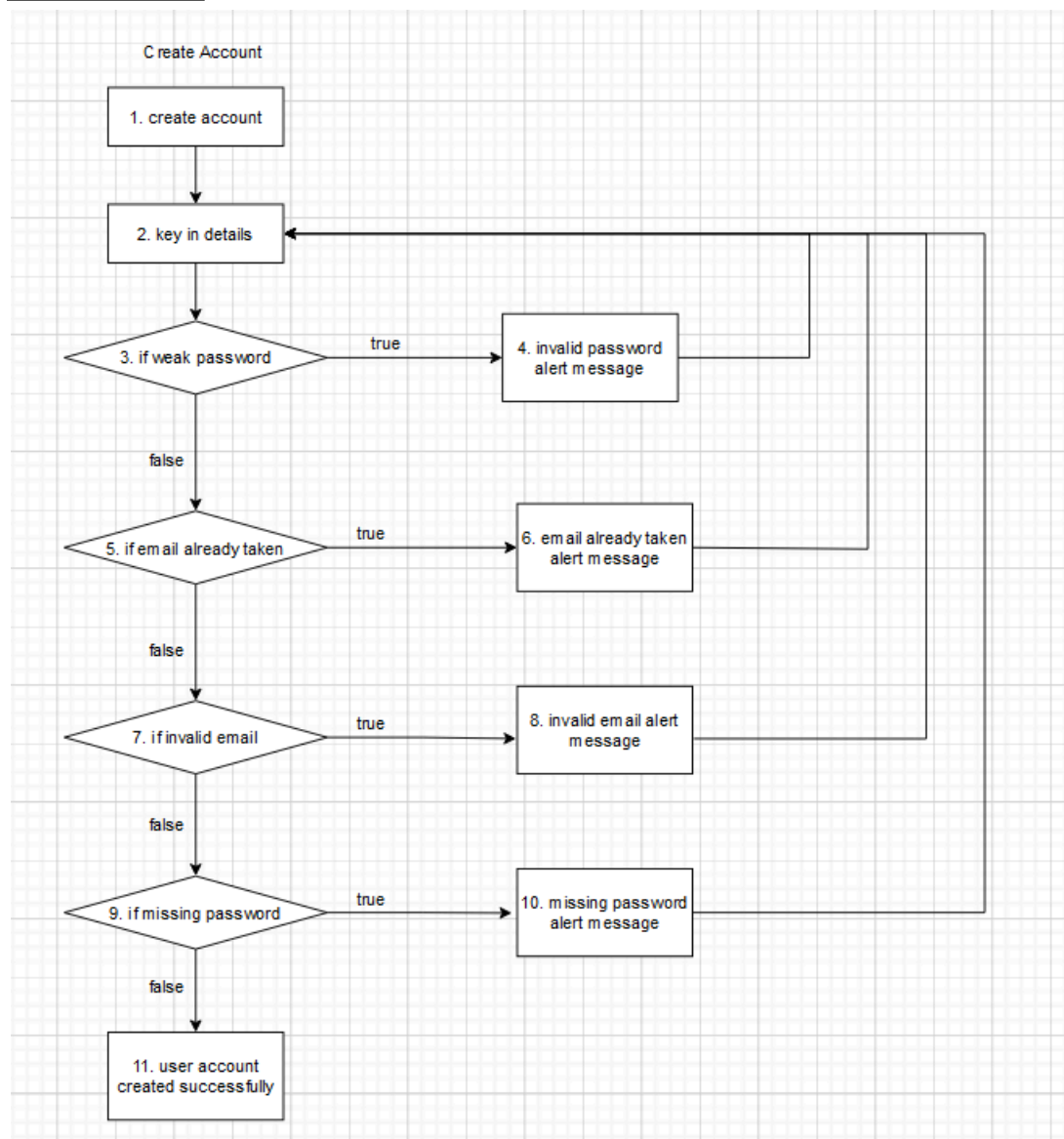
Email provided has string before and after '@' character	Email is not already in the database	Password has length greater than 5 and less than 30 characters	Password in "Password" and "Confirm Password" matches	Expected Output	Actual Output
Yes	Yes	Yes	Yes	Account is created and the user is directed to sign in.	Account is created and the user is directed to sign in.
No	Yes	Yes	Yes	"The email is invalid, please key in a valid email" message is sent.	"The email is invalid, please key in a valid email" message is sent.
Yes	No	Yes	Yes	"The email is already in use, please key in a different email." message is sent.	"The email is already in use, please key in a different email." message is sent.
Yes	Yes	No	Yes	"The password is too weak, please key in at least 6 characters." message is sent.	"The password is too weak, please key in at least 6 characters." message is sent.
Yes	Yes	Yes	No	"Passwords do not	"Passwords do not

				match. Please try again." message is sent.	match. Please try again." message is sent.
--	--	--	--	--	--

White Box Testing

Design test cases using basis path testing techniques to test two methods that implement complex application logic.

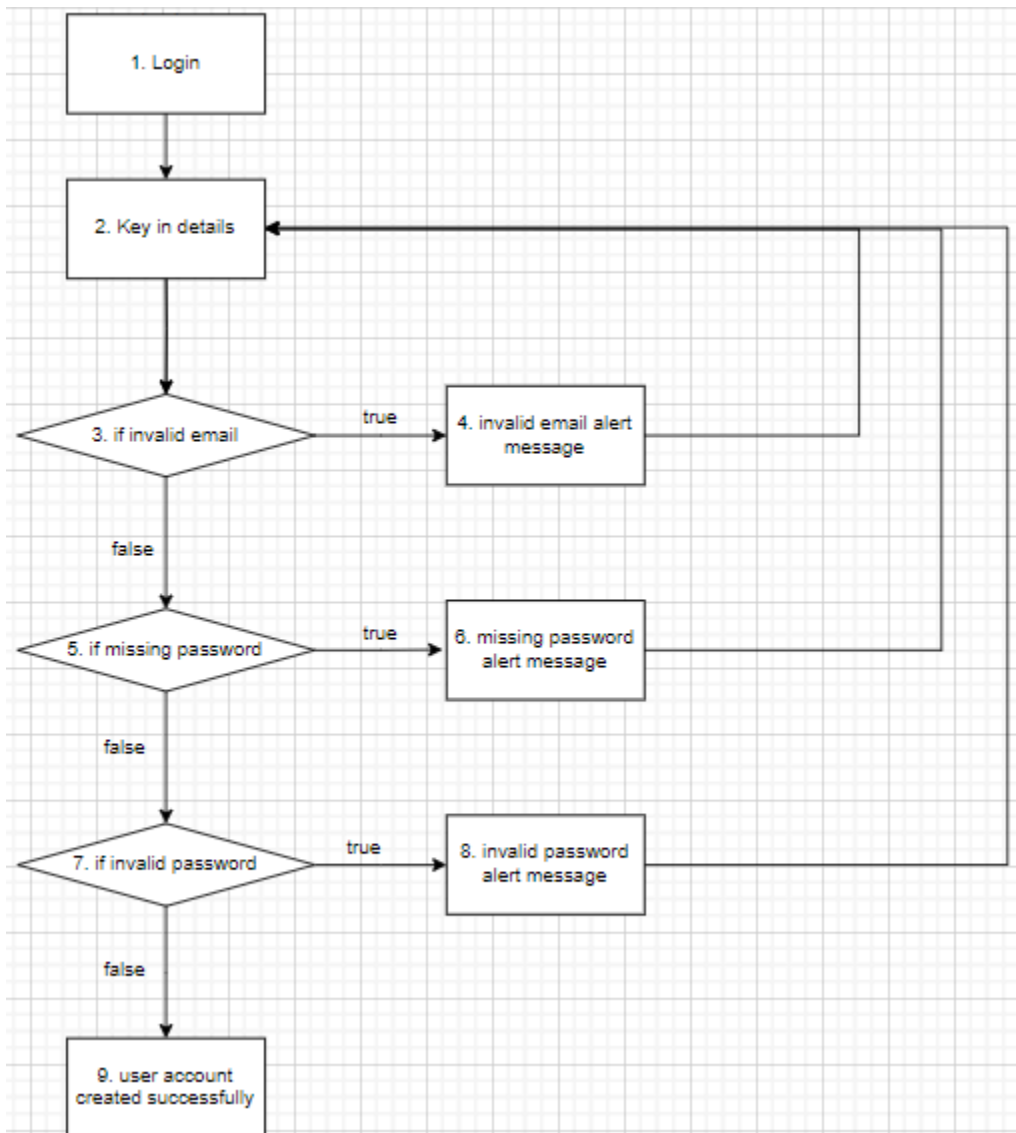
Create Account



Cyclomatic complexity = 5 basis paths

Basic Path	Test Input	Expected Output	Actual Output	Passed
1, 2, 3, 5, 7, 9, 11	weak_password = false; email_already_taken = false; invalid_email = false; missing_password = false;	User account created	User account created	Pass
1, 2, 3, 4, 2, 3, 5, 7, 9, 11	weak_password = true; email_already_taken = false; invalid_email = false; missing_password = false;	User will receive an alert to re-enter a stronger password.	User will receive an alert to re-enter a stronger password.	Pass
1, 2, 3, 5, 6, 2, 3, 5, 7, 9, 11	weak_password = false; email_already_taken = true; invalid_email = false; missing_password = false;	User will receive an alert to re-enter a new email.	User will receive an alert to re-enter a new email.	Pass
1, 2, 3, 5, 7, 8, 2, 3, 5, 7, 9, 11	weak_password = false; email_already_taken = false; invalid_email = true; missing_password = false;	User will receive an alert to re-enter a valid email.	User will receive an alert to re-enter a valid email.	Pass
1, 2, 3, 2, 5, 7, 9, 10, 2, 3, 5, 7, 9, 11	weak_password = false; email_already_taken = false; invalid_email = false; missing_password = true;	User will receive an alert to enter a password.	User will receive an alert to enter a password.	Pass

Login



Cyclomatic complexity = 4 basis paths

Basic Path	Test Input	Expected Output	Actual Output	Passed
1, 2, 3, 5, 7, 9	invalid_email = false; missing_password = false; invalid_password = false;	User account created	User account created	Pass
1, 2, 3, 4, 2, 3, 5, 7, 9	invalid_email = true; missing_password = false; invalid_password = false;	User will receive an alert to re-enter a valid email.	User will receive an alert to re-enter a valid email.	Pass
1, 2, 3, 5, 6, 2, 3, 5, 7, 9	invalid_email = false; missing_password = true; invalid_password = false;	User will receive an alert to enter a password.	User will receive an alert to enter a password.	Pass
1, 2, 3, 5, 7, 8, 2, 3, 5, 7, 9	invalid_email = false; missing_password = false; invalid_password = true;	User will receive an alert to re-enter a valid password.	User will receive an alert to re-enter a valid password.	Pass

9.6 Demo Script:

(Slide 1)

Hello everyone, my name is Seungju from team CodeCrafters and today we will be sharing our web application “GoWhere”.

(Slide 2)

In the first part of today’s presentation, we will be sharing our motivations and objectives of our project, followed by a live demonstration of GoWhere.

During the second portion, we will delve into a comprehensive analysis of project progress, leveraging software engineering concepts and diagrams.

(Slide 3)

The idea of our project emerged from our personal experiences with online place recommendations.

We have found ourselves bombarded with countless online recommendations for places to visit, only to end up repeatedly choosing the same familiar spots or relying on suggestions from our family and friends.

Although we often have specific categories of places in mind that we’d like to explore, our knowledge of new and exciting destinations is often limited.

This led us to ponder: How can we simplify the process of discovering new places, making it more enjoyable and reducing the hesitation that often accompanies trying something different?

(Slide 4)

In response to our question, we created GoWhere, a web application that helps users in Singapore to explore new places around the country.

GoWhere uses APIs for functions including weather, map, and place review, filtering places to recommend based on selected user preferences.

(Slide 5)

GoWhere achieves three main goals: Convenience, Entertainment, and Stress-Free exploration. Users get randomly selected recommendations with a streamlined experience, bypassing online clutter. The platform adds surprise and adventure to exploring new places, making it enjoyable. By curating suggestions and removing decision-making pressure, GoWhere ensures stress-free exploration, promoting spontaneity and confidence in discovering exciting destinations. Our targeted users range across all age groups, particularly those familiar with map applications like Google Maps.

(Slide 6)

To fulfill these objectives, our focus has centered on developing two primary functionalities: Account Management and Random Place Search. Account management is handled by the database, while place search is facilitated through APIs, as illustrated in the diagram. Consequently, our live demonstration will be divided into two parts, showcasing each of these functionalities separately.

(2 minutes and 5 seconds)

APP DEMONSTRATION

(Slide 8)

Creating GoWhere was definitely not an easy task, but by using an agile development method, we were able to minimize errors and develop in multiple iterations. Comparatively to the traditional waterfall model which lacks flexibility, an agile approach values short bursts of work, also known as sprints. Through multiple sprints, we were able to adapt to different directions, incorporating new information even at a later stage of the project.

Because of our limited experience, we were more likely to make mistakes and overlook certain details during the planning phase. By setting short-term deadlines, we were able to improve efficiency and collaborative work since we frequently share progress and feedback into the process.

(Slide 9)

The next slide shows our project system design, in which we segmented into 5 stages.

(Slide 10)

Starting from our homepage, the user will access the UI objects that are operated by five different managers.

(Slide 11)

These managers are linked to application objects, which are provided by APIs except for UserList, which is stored by the User Database.

(Slide 12)

The integration of agile development with segmented system architecture enabled a development environment conducive to continuous software improvement and evolution.

Segmented system architecture enhances modularity and scalability by dividing the system into independent, reusable modules. This division allowed us for future upgrades through modifications or replacements of specific segments without disrupting the entire system. Agile practices complement this approach by promoting iterative development and refactoring, empowering our team to enhance or replace modules incrementally.

Furthermore, agile development supports continuous integration and deployment (also known as CI/CD), allowing rapid and automated testing, integration, and deployment of changes. Segmented system architecture facilitates this by localizing changes within specific segments or modules, enabling independent integration and deployment. This streamlined process accelerates upgrade rollout and mitigates regression risks.

Additionally, agile development promotes proactive management of technical debt through continuous refactoring and software maintenance prioritization. Segmented system architecture aids in easier maintenance by isolating changes within specific segments, reducing the risk of system-wide issues. This incremental approach improved overall system maintainability and lays a strong foundation for further upgrades

(Slide 13)

To highlight our project traceability, we have brought the use case of saving and editing places.

Firstly, a user can use a bookmark button to edit the list of saved locations.

Upon clicking the button on the Map page, the system manager sends a request to the database to add place details.

Once stored, it will then be available to be viewed on the “Saved Locations” page.

If the user un-clicks the button on the Saved Locations page: the system manager requests the database again to remove the place and its details.

(Slide 14)

Based on this use case, here is the sequence diagram we have conducted. Through 3 boundary objects and one control object, a location is saved and removed through four procedures each, which eventually displays the result back to the user.

(Slide 15)

The design we applied in this use case is the observer pattern, which involves one subject, the BookMark Manager, and two observers BookMarkUI & Saved Locations Page, excluding Database which acts as a separate component, responsible for persisting and managing user-related data.

The contents on right shows how the Observer Pattern is applied.

- Notification Mechanism: When a user adds or removes a bookmarked location through the BookMarkUI, the BookMark Manager (subject) updates its internal state.
- Automatic Update: After updating its state, the BookMark Manager notifies all registered observers (BookMarkUI and Saved Locations page) about the change.
- Decoupling: This pattern helps in decoupling the BookMarkUI and Saved Locations page from the internal details of the BookMark Manager. Observers don't need to know how the bookmarks are managed internally; they only care about being notified of changes.

(Slide 16)

The benefits of such design is that there is a clear separation of concerns between components involved in managing and displaying bookmarked locations.

The UI components (BookMarkUI and Saved Locations Page) remain decoupled from data storage logic. On top of this, the BookMark Manager orchestrates updates while the User Database manages data persistence seamlessly.

In summary, by leveraging the Observer Pattern, our project achieves enhanced modularity, flexibility and maintainability in managing bookmarked locations.

(Slide 17)

To wrap up our presentation, this project has provided a valuable learning experience in the field of software engineering. Throughout our journey, we have made significant improvements and gained several key takeaways. Individually, each team member has acquired essential technical skills for software development, contributing to our personal growth. Collectively, we have learned valuable lessons in optimizing collaborative work and efficient task management, enhancing our abilities as a cohesive team.(Slide 1)